DTU [Department]
Department of [...]

Embedding Electricity Price Forecasting Benchmarks
in a Production MLOps Platform

Bachelor’s Thesis



Embedding Electricity Price Forecasting Benchmarks
in a Production MLOps Platform

Bachelor's Thesis
June 2026

By
[Author Name]

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.
Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of [...], [Building / street], 2800 Kgs. Lyngby Denmark
www.dtu.dk
ISSN: [0000-0000] (electronic version)
ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)
ISBN: [000-00-0000-000-0] (printed version)

Preface

This thesis was prepared at [Department], the Technical University of Denmark (DTU), in partial fulfilment of the requirements for the degree of Bachelor of Science in Engineering.

[Author Name] - [s000000]

.....

Signature

.....

Date

Abstract

Research on day-ahead electricity price forecasting (EPF) has produced accurate benchmark models, but it evaluates them offline. It stops short of the concerns that arise when a model runs in production, such as scheduled retraining, monitoring, and reproducibility. This thesis addresses that gap. It embeds the benchmark EPF models of Lago et al. (the LEAR ensemble and a deep neural network) and the Transformer of Llorente and Portela in an end-to-end MLOps platform, and it evaluates both the forecasts and the system that serves them. The platform ingests ENTSO-E data on a schedule, stores it in ClickHouse, recalibrates and forecasts daily for the DK1 and DE bidding zones, monitors data and model health, and records the provenance of every run. The three models are replicated to specification and compared on a shared recent test window with paired Diebold–Mariano and Giacomini–White significance tests. LEAR and a tuned DNN are statistically tied as the most accurate models on both markets ($p > 0.16$), both significantly outperform the pure Transformer ($p \leq 0.012$), and all three beat the weekly-naive baseline. The source papers' model rankings do not replicate under the reduced tuning budget used here. The system is assessed against the nine MLOps practices of Zarour et al. Eight are realised end-to-end and one is minimally realised. The assessment is supported by an ML Test Score of 2.5 bound by data testing, and 66.7 % provenance-triple coverage backed by a re-run that reproduced a logged forecast to within 7×10^{-6} EUR/MWh. The result is a reproducible, monitored EPF platform and a faithful, significance-tested model comparison on current market data.

Acknowledgements

[Acknowledgements to be added.]

Contents

Preface . . . ii
Abstract . . . iii
Acknowledgements . . . iv
Acronyms . . . ix

1 Introduction **1**
1.1 Motivation . . . 1
1.2 The gap between forecasting research and operation . . . 1
1.3 Aim and research questions . . . 1
1.4 Contributions . . . 2
1.5 Report structure . . . 2

2 Background and Related Work **3**
2.1 Day-ahead electricity price forecasting . . . 3
2.2 The Lago et al. open-access benchmark . . . 3
2.3 Transformer approaches to EPF . . . 4
2.4 MLOps: practices and maturity . . . 4
2.5 Evaluating machine-learning systems . . . 4

3 Methods: Forecasting Models and Evaluation **5**
3.1 Replication framing . . . 5
3.2 Data and features . . . 5
3.3 Models . . . 6
3.4 Recalibration and backtesting . . . 10
3.5 Evaluation protocol . . . 10
3.6 Hyperparameter tuning . . . 11
3.7 Deviations from the source papers . . . 11

4 System Implementation **13**
4.1 Scope and approach . . . 13
4.2 System at a Glance . . . 13
4.3 Implementing Zarour's Nine MLOps Practices . . . 16
4.4 Reproducibility, monitoring, and single-host scope . . . 22

5 Results: Forecasting Models **24**
5.1 Headline accuracy . . . 24
5.2 Statistical significance . . . 25
5.3 Computation cost . . . 25
5.4 Comparison with published benchmarks . . . 26

6 Results: System Evaluation **27**
6.1 ML Test Score . . . 27
6.2 Reproducibility . . . 28
6.3 Drift detection as monitoring, not a retraining trigger . . . 29
6.4 Maturity placement . . . 29

7 Discussion **31**
7.1 RQ1: Comparing the forecasting models 31
7.2 RQ2: Engineering trade-offs of productionising an EPF benchmark 32
7.3 RQ3: Critical practices and the limits of a single-host platform 32
7.4 Tying the threads together 32
7.5 Limitations 33

8 Conclusion and Future Work **34**
8.1 Future work 34

Bibliography **37**

A Glossary **39**

B ML Test Score Worksheet **40**

List of Figures

3.1 Reshaping from the stored long (time-series) form, one row per hour, to the daily-wide layout the models consume, one row per day with the $n{=}24$ hourly prices as columns ($h_1, \ldots, h_{24}$). Lagged price and exogenous blocks are built by joining date-shifted copies of these wide rows. 6

3.2 The DNN, after Lago et al. [3], Fig. 4: a fully-connected multilayer perceptron from the feature vector of Eq. (3.1) to the n delivery periods. The two hidden layers share a single tuned width (Table 3.4: 191 for DK1, 373 for DE); the number of units drawn is schematic. 8

3.3 The encoder-only Transformer, after Llorente and Portela [6], Fig. 2a. The past-price sequence is embedded and given a learnable positional encoding before the encoder stack; the day-ahead exogenous forecasts are embedded in parallel; the two representations are concatenated and mapped to the n output periods by an MLP. 9

3.4 Rolling walk-forward recalibration with a calibration-window ensemble, after Lago et al. [3]. For each test day the data are sliced to the most recent window of each ensemble length (shown for LEAR: 56/84/1092 days), every window is refit and predicts the target day, and the forecast is their average. The boundary then advances one day; models refit daily (LEAR, DNN) or weekly (Transformer). 10

4.1 Host-level architecture. A single Docker Compose host runs the six long-lived services, namely ClickHouse, the Prefect server and worker, Prometheus, Grafana, and Loki. Flow runs spawn ephemeral `algo-repo:latest` containers that read and write ClickHouse, fetch from ENTSO-E, and push logs to Loki; Prometheus scrapes service metrics and Grafana queries Prometheus and Loki. 14

4.2 The CI/CD pipeline. On a pull request, three checks (`pr_test`, `pr_ruff-lint`, `pr_smoke`) must pass before a merge. On merge to `main`, `cd_deploy` reaches the host over Tailscale, rebuilds the image, redeploys the Prefect flows, and runs a smoke forecast. 17

4.3 The daily train-and-evaluate cadence (UTC). At 06:00 every configuration refits from scratch and produces a day-ahead forecast; at 18:00 reconciliation joins the published actuals to the stored predictions and scores them; at 19:00 drift detection runs. The cycle repeats every day. 19

4.4 Data plane (high level). ENTSO-E data is ingested into ClickHouse; the forecasting pipeline loads it, builds features, and trains and serves the models, writing predictions and artifacts back. The daily reconciliation-and-drift loop reads those predictions, writes back actuals and metrics, and raises alerts or retraining. 22

List of Tables

3.1 Datasets. Both markets span 2022-03-10 to 2026-05-31 (ENTSO-E Transparency Platform). . . . 5
3.2 Feature count by lag at hourly resolution (n=24), itemising Eq. (3.1). Every lagged series contributes one column per delivery period ($= n$), so $10 \times 24 + 7 = 247$. . . . 7
3.3 Model setups (one row per model, identical across both markets unless noted). . . . 8
3.4 Tuned DNN hyperparameters per market. Selected by the protocol of §3.6 on the ex-ante validation window. The architecture-only search is re-run whenever the feature set changes, so these values are refreshed by the paper-exogenous re-tune that accompanies the final benchmark. . . . 9
3.5 Documented deviations from [3] and [6]. . . . 12

4.1 Operational inventory of Prefect deployments. Cron strings are UTC; the seven backfill deployments and six forecast deployments share one flow definition apiece. . . . 15
4.2 Technology in use and its role. The rationale for each choice is given in the practice section where the tool appears; tools evaluated but not adopted (MLflow, Kubernetes, DVC) are discussed there too. . . . 16
4.3 Monitoring coverage. The first three rows are Zarour's P8 axes; *Operational* is the shared structured-log stream beneath them. Grafana and W&B both read from ClickHouse. . . . 21
4.4 Alert rules and their severities. The first four are file-provisioned Grafana rules in `grafana/provisioning/alerting/rules.yaml`, evaluated over ClickHouse except `clickhouse-down`, which reads the Prometheus liveness metric. `accuracy-degradation` is a Slack check inside the reconciliation flow. . . . 21

5.1 DK1: final benchmark metrics (test 2025-10-24–2026-05-31, common days). 24
5.2 DE: final benchmark metrics (test 2025-10-24–2026-05-31, common days). 24
5.3 DK1: GW multivariate p-values (column model better than row model). . . . 25
5.4 DE: GW multivariate p-values (column model better than row model). . . . 25
5.5 Amortised recalibration cost (full-backtest wall-clock / common test day). Full timings in `Report/metrics/run_timings.md`. . . . 26

6.1 ML Test Score self-assessment (min-of-sections rule). Per-item evidence in Appendix B. . . . 27
6.2 Provenance triple coverage across 15 live runs in the window. . . . 28
6.3 Reproducibility re-run: regenerated from logged provenance, compared against the stored prediction. . . . 28
6.4 Maturity assessment of Zarour's nine practices, with the Google MLOps maturity model level [10] assigned to each. . . . 30

B.1 Section 1: Data tests (sum 2.5 / 7). . . . 40
B.2 Section 2: Model development tests (sum 3.0 / 7). . . . 40
B.3 Section 3: ML infrastructure tests (sum 4.0 / 7). . . . 41
B.4 Section 4: Monitoring tests (sum 4.0 / 7). . . . 41

Acronyms

CI/CD Continuous Integration / Continuous Delivery

DM Diebold–Mariano (test of equal predictive accuracy)

DNN Deep Neural Network

ENTSO-E European Network of Transmission System Operators for Electricity

EPF Electricity Price Forecasting

GW Giacomini–White (test of conditional predictive ability)

LEAR Lasso Estimated AutoRegressive (model)

MAE Mean Absolute Error

MLOps Machine Learning Operations

rMAE relative Mean Absolute Error

RMSE Root Mean Squared Error

sMAPE symmetric Mean Absolute Percentage Error

TPE Tree-structured Parzen Estimator

VST Variance-Stabilising Transformation

1 Introduction

1.1 Motivation

Day-ahead electricity price forecasting (EPF) is the task of predicting the wholesale price of electricity for each delivery period of the following day in a given bidding zone, from a combination of historical prices and exogenous signals such as load and renewable-generation forecasts. Accurate day-ahead forecasts are essential to generators, retailers, and grid operators, who rely on them to schedule production, hedge positions, and bid into the day-ahead auction. The task has grown harder over the past decade. The expansion of wind and solar generation introduces supply-side volatility that statistical models struggle to capture, and the coupling of European bidding zones propagates price shocks across borders in ways historical data may not reflect. The resulting price series are heavy-tailed and spike-prone. Under high renewable infeed they increasingly admit negative prices, a regime that rules out modelling choices that presume positivity, such as a logarithmic price transform [1].

1.2 The gap between forecasting research and operation

Research on EPF has produced increasingly accurate models, spanning statistical, reduced-form, and machine-learning approaches, and the field has been surveyed in depth [1, 2]. The open-access benchmark of Lago et al. [3] was an important step. It documented how severe the evaluation problem had become. New methods were compared on private datasets, over test periods too short to be conclusive, and against baselines too weak to be informative. In response, Lago et al. standardised both a pair of benchmark models and a methodology for comparing them.

Yet even the most careful EPF contributions stop at the boundary of offline evaluation. They standardise how models are *compared*, but not how they are *run*. Ingesting data on a schedule, versioning experiments, detecting model degradation, and retraining on fresh data are left unaddressed. This gap between building a model and operating it is the one that Machine Learning Operations (MLOps) emerged to close, by carrying DevOps principles such as automation, reproducibility, and monitoring into machine-learning systems. Zarour et al. [4] synthesise the MLOps literature into a systematic review of nine core practices and an accompanying maturity model. The two bodies of work rarely meet. EPF research treats forecasting as a problem that ends when predictions are produced, while MLOps research is largely domain-agnostic. The closest precedent for joining them is the work of Subramanya et al. [5], who apply an MLOps pipeline to electricity-market forecasting as a case study.

1.3 Aim and research questions

This thesis takes the benchmark EPF models from the forecasting literature and embeds them in an end-to-end production platform built along MLOps principles. The platform ingests data from the ENTSO-E Transparency Platform on a recurring schedule, stores it in ClickHouse, produces day-ahead forecasts through rolling-window recalibration, monitors prediction quality and data drift, and maintains a complete audit trail from every forecast back to the code version and data window that produced it. It is evaluated on two axes: the *forecast quality* of the models it serves, and the *operational maturity* of the system that serves them. Three research questions structure the investigation.

RQ1 (forecasting). How do faithfully replicated LEAR, DNN, and Transformer day-ahead EPF models compare on recent ENTSO-E data for the DK1 and DE bidding zones, when set against one another, a weekly-naive baseline, and the rankings reported by Lago et al. [3] and Llorente and Portela [6]?

RQ2 (engineering). What engineering trade-offs and operational constraints emerge when embedding these benchmark models into a scheduled, containerised, continuously monitored production pipeline?

RQ3 (maturity). Assessed against the nine MLOps practices of Zarour et al. [4] and the Google maturity model, which practices does the platform realise, which prove most critical, and where does a single-host minimum-viable platform reach its limit?

1.4 Contributions

This work makes four contributions.

- **A production EPF platform.** An end-to-end system that ingests live ENTSO-E data, recalibrates and forecasts on a daily schedule, monitors data and model health, and records the full provenance of every run. This is the artefact that RQ2 and RQ3 are evaluated against.
- **A faithful model replication.** LEAR and the DNN of Lago et al. [3] and the Transformer of Llorente and Portela [6], reimplemented to specification and compared across two markets on a shared recent test window with paired Diebold–Mariano and Giacomini–White significance tests (RQ1).
- **A composed quantitative MLOps evaluation.** A combination of the ML Test Score rubric, provenance-coverage measurement, and a worked reproducibility re-run, assembled to give the otherwise-qualitative maturity question a measurable answer.
- **A practice-by-practice maturity assessment** of the platform against Zarour et al.'s nine practices, with each practice traced to a concrete repository artefact and positioned on the Google maturity model.

The scope is deliberately bounded. The work forecasts point prices, not probabilistic distributions. It is a single-case study of one platform, so its system-side findings support analytic rather than statistical generalisation. Because the source papers' test periods predate the present price regime, the forecasting comparison is framed in relative terms (rMAE and rankings) and not as a comparison of absolute errors.

1.5 Report structure

Chapter 2 reviews the EPF and MLOps literatures and the evaluation methodology this work draws on. Chapter 3 specifies the three forecasting models, their features, the recalibration scheme, and the metrics and significance tests used to compare them. Chapter 4 documents the production system around those models, organised around Zarour's nine practices. Chapter 5 reports the forecasting comparison, and Chapter 6 sets out how the system is evaluated and reports the result. Chapter 7 answers the three research questions and draws the two threads together, and Chapter 8 concludes and identifies future work.

2 Background and Related Work

This chapter reviews the two literatures the thesis joins, electricity price forecasting and MLOps, together with the evaluation methodology it borrows from empirical software engineering. Section 2.1 covers day-ahead EPF and its statistical character. Section 2.2 introduces the open-access benchmark that anchors the modelling thread. Section 2.3 describes the Transformer approach that supplies the third model. Section 2.4 sets out the MLOps practices and maturity models behind the system thread. Section 2.5 lists the instruments used to evaluate the platform.

2.1 Day-ahead electricity price forecasting

In a day-ahead market, participants submit bids for each delivery period of the following day and a uniform clearing price is set, once per day, at the intersection of aggregated supply and demand within each bidding zone. The clearing price is set by the marginal generator, so it inherits the cost structure of the generation mix and the topology of cross-border exchange. This mechanism produces a price series with pronounced multi-scale seasonality (daily, weekly, and annual), heavy tails, and sudden spikes driven by scarcity or congestion. As wind and solar capacity has grown, two further features have become routine: sharp drops, and outright negative prices, when inflexible must-run generation coincides with high renewable infeed and low demand. Weron's review [1] surveys the statistical and computational-intelligence methods developed for this problem, and Nowotarski and Weron [2] extend the picture to probabilistic forecasting. Both emphasise that the EPF methodology, and not only the choice of model, determines whether reported gains are real.

2.2 The Lago et al. open-access benchmark

Lago et al. [3] address the reproducibility problem in EPF directly, contributing an open-access benchmark comprising five market datasets (Nord Pool, PJM, and the EPEX zones BE, FR, and DE), two reference models, and the `epftoolbox` library that standardises data splits and evaluation. The first reference model, LEAR (Lasso Estimated AutoRegressive), fits an L^1-regularised linear model over a parameter-rich feature space of price lags, exogenous forecasts, and calendar dummies. The Lasso performs feature selection and coefficient estimation jointly, which keeps the rich input space tractable. The second is a deep neural network, a multi-layer perceptron whose architecture and inputs are tuned per dataset. Despite its simplicity, LEAR matched or outperformed the deep alternative when evaluated over long test periods, which is why it serves as the modelling foundation of this thesis.

Two methodological contributions of the benchmark are reused here directly. First, models are recalibrated on a rolling window and their forecasts averaged across an ensemble of window lengths, rather than trained once. Second, the benchmark argues for the relative mean absolute error (rMAE), which normalises the model's error by that of a naive baseline so that results remain comparable across periods and markets, and for formal significance testing of accuracy differences via the Diebold–Mariano [7] and multivariate Giacomini–White [8] tests. The benchmark also adopts the inverse-hyperbolic-sine (asinh) variance-stabilising transform of Uniejewski et al. [9], which compresses spikes while remaining defined for negative prices.

2.3 Transformer approaches to EPF

Attention-based architectures have more recently been applied to EPF. Llorente and Portela [6] propose an encoder-only Transformer. It takes two inputs, a sequence of the previous days' prices and the day-ahead exogenous forecasts. It embeds and attends over the price sequence, then combines the result with the exogenous representation through a final multi-layer perceptron. Benchmarked on the same open-access datasets, it provides the third model family compared in this work. The authors evaluate without retraining inside the test period, applying only a renormalisation step between days. This thesis instead places the Transformer in the same rolling-recalibration pipeline as the other models, a deviation documented in Chapter 3.

2.4 MLOps: practices and maturity

MLOps applies DevOps principles such as automation, version control, continuous delivery, and monitoring to the particular demands of machine-learning systems, where data and models, not only code, evolve over time. Zarour et al. [4] consolidate the field in a systematic literature review that identifies nine recurring best practices: automating CI/CD, orchestrating workflows, ensuring reproducibility, versioning, collaboration and communication, continuously training and evaluating models, tracking and logging ML metadata, continuous monitoring, and implementing feedback loops. These nine practices provide the organising structure for the system chapter (Chapter 4).

The same review surveys five MLOps maturity models, all descended from the CMM/CMMI lineage: the Google three-level model, the Microsoft and GigaOm five-level models, the four-level model of John, Olsson and Bosch, and the five-level model of Mateo-Casalí et al. This work positions the platform against the Google model [10], whose three levels (manual process, ML-pipeline automation, and CI/CD-pipeline automation) match the granularity of a single-host platform. That choice is justified in Chapter 6. The closest domain application is Subramanya et al. [5], who build an MLOps pipeline for an electricity-market forecasting task and evaluate it as a case study. This is the precedent this thesis follows in evaluating its own platform in its real operating context.

2.5 Evaluating machine-learning systems

Because the platform is a single artefact rather than a sample, its system-side evaluation follows the single-case study methodology of Runeson and Höst [11]. That method supports *analytic* generalisation, extending findings to cases that share the platform's characteristics, rather than statistical generalisation to a population. Within that frame, several established instruments are composed. The ML Test Score of Breck et al. [12] is a 28-item rubric across four sections (data, model development, ML infrastructure, and monitoring) whose final score is the minimum of the four section sums, so the weakest area bounds the result. Reproducibility is treated in two senses. Gundersen and Kjensmo [13] characterise degrees of reproducibility in AI research. Finally, the decision of whether drift should trigger retraining is informed by the cost-aware analysis of Mahadevan and Mathioudakis [14], who show that triggered retraining earns its keep mainly when retraining is expensive. How these instruments are applied, and why each was chosen, is the subject of Chapter 6.

3 Methods: Forecasting Models and Evaluation

This chapter specifies the forecasting models that the platform serves and the protocol by which they are compared. Three model families are studied: the Lasso Estimated AutoRegressive ensemble (LEAR) and the deep neural network (DNN) of Lago et al. [3], and the encoder-only Transformer of Llorente and Portela [6]. Each is reimplemented faithfully and run on recent day-ahead data for two European bidding zones, DK1 (Nord Pool) and DE (EPEX). The chapter first states what ”faithful replication” means and why the comparison is framed in relative rather than absolute terms (§3.1), then describes the data and feature construction (§3.2), the three models as implemented (§3.3), the rolling recalibration and backtesting scheme (§3.4), the evaluation metrics and significance tests (§3.5), the hyperparameter-tuning protocol (§3.6), and finally the documented deviations from the source papers (§3.7).

3.1 Replication framing

The error magnitudes reported in the source papers were obtained on 2016–2018 test periods whose price level and volatility differ sharply from the 2025–2026 regime studied here. An absolute mean-error figure is therefore not comparable across the two eras. The comparison this work makes has three parts. The first part is methodology replication. It uses the same model specifications, feature sets, recalibration scheme, metrics, and statistical tests as the papers, run on current data. The second part is relative accuracy, measured by the relative mean absolute error (rMAE), which Lago et al. [3] designed for cross-period and cross-market comparison, together with the ordinal ranking of the models against the rankings the papers report. The third part is head-to-head significance, using paired Diebold–Mariano [7] and Giacomini–White [8] tests across the three models and a weekly-naive baseline, per market, on identical test days. Because all six experimental runs (three models $\times$ two markets) share a single test window, the paired tests are valid: every pair of models is compared on the same delivery days.

3.2 Data and features

Table 3.1: Datasets. Both markets span 2022-03-10 to 2026-05-31 (ENTSO-E Transparency Platform).

Market	Resolution	Exogenous (paper-exact)	Paper analogue
DK1	hourly (24)[a]	TotalLoad + Wind (on+offshore)	Nord Pool: load + wind [3]
DE	hourly (24)[a]	TotalLoad + Renewables (wind+solar)	EPEX-DE: load + wind&solar [3]

[a] Both markets clear in 15-min blocks (DE throughout, DK1 since 2025-10-01), and prices and exogenous forecasts are downsampled to an arithmetic hourly mean (`resample_to_hourly`).

All data are obtained from the ENTSO-E Transparency Platform and stored in ClickHouse, from which the preprocessor loads a price series and a configurable set of exogenous

forecast series for each market. The data arrive in long, time-series form, one row per delivery period (an hour, or a quarter-hour before downsampling); the preprocessor pivots each series to the daily-wide layout of Fig. 3.1, from which the per-day feature vector is assembled.

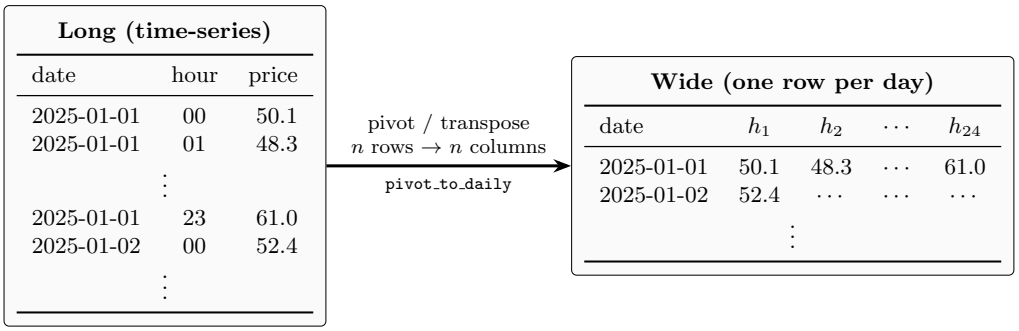


Figure 3.1: Reshaping from the stored long (time-series) form, one row per hour, to the daily-wide layout the models consume, one row per day with the $n{=}24$ hourly prices as columns ($h_1, \ldots, h_{24}$). Lagged price and exogenous blocks are built by joining date-shifted copies of these wide rows.

Following Lago et al. [3], Eq. 3, the per-day feature vector combines autoregressive price lags, lagged exogenous forecasts, and seven day-of-week dummies (itemised by lag in Table 3.2). For a day discretised into n delivery periods this yields

$$\underbrace{4n}_{\text{price lags}} + \underbrace{2 \cdot 3n}_{\text{two exogenous} \times \text{3 lags}} + \underbrace{7}_{\text{weekday}} \tag{3.1}$$

features, i.e. 247 at hourly resolution ($n{=}24$) and 967 at quarter-hourly resolution ($n{=}96$).

The exogenous set is restricted, per market, to exactly the two signals the papers use, the load and wind/renewables aggregates of Table 3.1. These aggregates are formed at load time by summing the raw ENTSO-E curves: the member curves are inner-joined on their timestamps before summing so a partial sum can never occur, and all exogenous columns are then inner-joined together, so the coarsest available series anchors the common time grid.

The target undergoes the asinh variance-stabilising transform of Uniejewski et al. [9], which compresses the heavy tails and price spikes characteristic of day-ahead series while, unlike a logarithm, remaining defined for the negative prices that occur under high renewable infeed. Prices and exogenous forecasts are downsampled to an arithmetic hourly mean before features are constructed (Table 3.1), which keeps the two markets comparable with each other and with the hourly reference benchmarks of Lago et al. [3] and Llorente and Portela [6]; modelling DK1 natively at 96 periods is in any case infeasible, with under one month of quarter-hourly history before the test window. Finally, ENTSO-E lacks DK1 wind forecasts for a small number of days inside the sample; those days are dropped identically from every DK1 model, so the paired significance tests remain valid.

3.3 Models

Table 3.3 summarises the three model setups, which are identical across both markets except for the market-specific exogenous aggregate and the per-market DNN hyperparameters.

Table 3.2: Feature count by lag at hourly resolution (n=24), itemising Eq. (3.1). Every lagged series contributes one column per delivery period ($= n$), so $10 \times 24 + 7 = 247$.

Feature block	**Lag**	**# columns ($= n$)**
Price p	$d{-}1$	24
Price p	$d{-}2$	24
Price p	$d{-}3$	24
Price p	$d{-}7$	24
Exogenous x^1	$d0$	24
Exogenous x^1	$d{-}1$	24
Exogenous x^1	$d{-}7$	24
Exogenous x^2	$d0$	24
Exogenous x^2	$d{-}1$	24
Exogenous x^2	$d{-}7$	24
Weekday dummies z	n/a	7
Total		**247**

3.3.1 LEAR

LEAR is a direct multi-period regressor: one independent Lasso model is fitted per delivery period, so an hourly market trains n=24 separate estimators, each predicting a single period from the full feature vector of Eq. (3.1). Writing x^1, x^2 for the two exogenous series and z_d for the seven day-of-week dummies, the (asinh-transformed) price for period h is modelled, following Lago et al. [3], Eq. 3, as a linear combination of the price lags, the exogenous lags, and the weekday dummies:

$$\begin{aligned}\hat{p}_{d,h} = &\sum_{i=1}^{n} \theta_{h,i}\, p_{d-1,i} + \sum_{i=1}^{n} \theta_{h,n+i}\, p_{d-2,i} + \sum_{i=1}^{n} \theta_{h,2n+i}\, p_{d-3,i} + \sum_{i=1}^{n} \theta_{h,3n+i}\, p_{d-7,i} \\ &+ \sum_{i=1}^{n} \theta_{h,4n+i}\, x^1_{d,i} + \sum_{i=1}^{n} \theta_{h,5n+i}\, x^2_{d,i} + \sum_{i=1}^{n} \theta_{h,6n+i}\, x^1_{d-1,i} + \sum_{i=1}^{n} \theta_{h,7n+i}\, x^2_{d-1,i} \\ &+ \sum_{i=1}^{n} \theta_{h,8n+i}\, x^1_{d-7,i} + \sum_{i=1}^{n} \theta_{h,9n+i}\, x^2_{d-7,i} + \sum_{j=1}^{7} \theta_{h,10n+j}\, z_{d,j},\end{aligned} \tag{3.2}$$

where $\boldsymbol{\theta}_h \in \mathbb{R}^{10n+7}$ ($= \mathbb{R}^{247}$ at n=24) collects the coefficients for period h. They are estimated by least squares under an L^1 penalty (the LASSO of [3], Eq. 4),

$$\hat{\boldsymbol{\theta}}_h = \arg\min_{\boldsymbol{\theta}_h} \sum_{d} \left(p_{d,h} - \hat{p}_{d,h}\right)^2 + \lambda \left\|\boldsymbol{\theta}_h\right\|_1, \qquad \left\|\boldsymbol{\theta}_h\right\|_1 = \sum_{k=1}^{10n+7} |\theta_{h,k}|, \tag{3.3}$$

so the penalty performs simultaneous shrinkage and feature selection and keeps the parameter-rich input space tractable. The strength $\lambda \geq 0$ is selected automatically per period by `LassoCV` with a five-fold `TimeSeriesSplit` cross-validation (maximum 5000 coordinate-descent iterations), the configuration denoted `alpha="auto"`. LEAR carries

Table 3.3: Model setups (one row per model, identical across both markets unless noted).

Model	Specification	Calibration	Refit
LEAR	One Lasso per delivery period, λ by `LassoCV` (time-series CV)	ensemble over 56 / 84 / 1092 d	daily
DNN	MLP, architecture HPs by Optuna TPE (ex-ante validation), 4-seed ensemble at serving (single seed in backtest)	730 d	daily
Transformer	Encoder stack per [6], paper Nord Pool hyperparameters	1092 d	weekly

no learned ensembling of its own; the ensemble averaging it benefits from is supplied by the multi-window recalibration scheme of §3.4.

3.3.2 Deep neural network

The DNN is a multilayer perceptron with two hidden layers that maps the feature vector of Eq. (3.1) to all n delivery periods in a single forward pass, a multiple-input multiple-output design (Fig. 3.2).

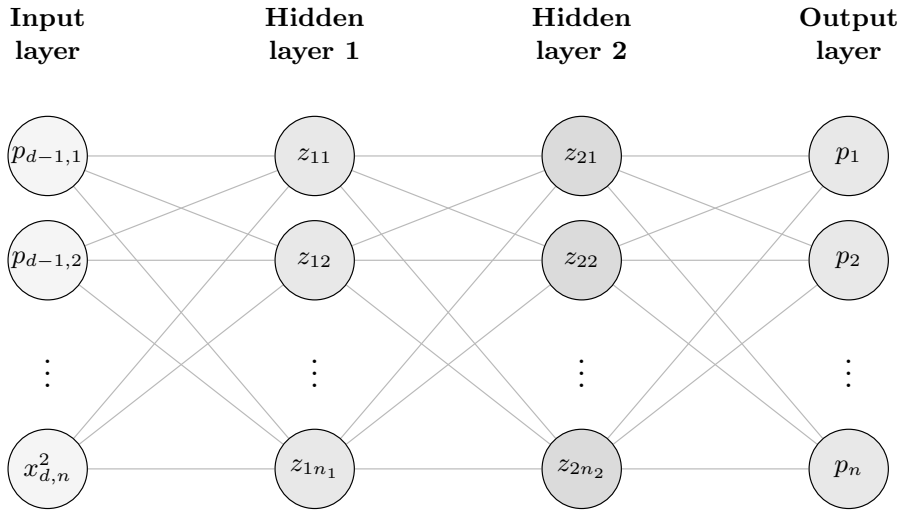


Figure 3.2: The DNN, after Lago et al. [3], Fig. 4: a fully-connected multilayer perceptron from the feature vector of Eq. (3.1) to the n delivery periods. The two hidden layers share a single tuned width (Table 3.4: 191 for DK1, 373 for DE); the number of units drawn is schematic.

It is trained to minimise the mean absolute error with the Adam optimiser, with optional L1 weight regularisation, optional batch normalisation, and early stopping (the best weights, by validation loss, restored from memory). The serving prediction is the arithmetic mean of four replicates trained from distinct random seeds, following Lago et al. [3], §4.4; this four-seed averaging is applied only on the production serving path. The rolling backtest behind the Chapter 5 results trains a single network per recalibration at a fixed seed, so the reported DNN metrics carry the seed variance of one draw. The six architecture hyperpa-

rameters (hidden-layer width, activation, dropout, learning rate, the batch-normalisation flag, and the L1 penalty) are selected per market by the tuning protocol of §3.6; Table 3.4 lists the values in force.

Table 3.4: Tuned DNN hyperparameters per market. Selected by the protocol of §3.6 on the ex-ante validation window. The architecture-only search is re-run whenever the feature set changes, so these values are refreshed by the paper-exogenous re-tune that accompanies the final benchmark.

Market	Width	Activation	Dropout	Learning rate	Batch norm	L1 penalty
DK1	191	tanh	0.203	4.8×10^{-4}	no	6.8×10^{-8}
DE	373	relu	0.204	6.9×10^{-4}	yes	7.1×10^{-7}

3.3.3 Transformer

The Transformer follows Llorente and Portela [6] and is encoder-only with two parallel input branches (Fig. 3.3).

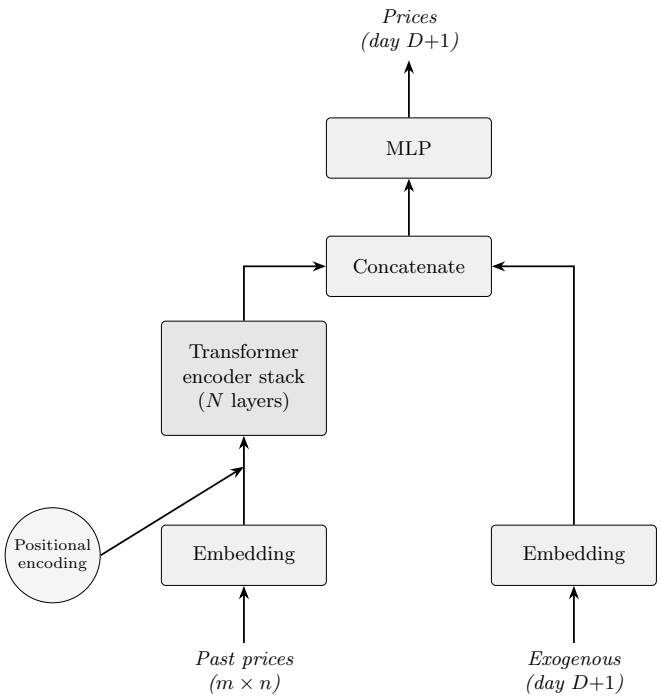


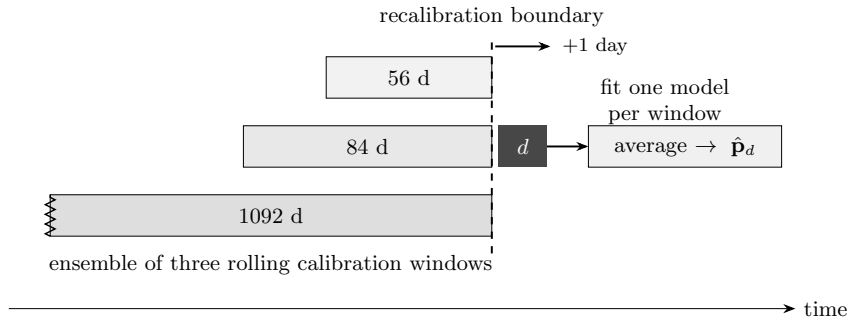
Figure 3.3: The encoder-only Transformer, after Llorente and Portela [6], Fig. 2a. The past-price sequence is embedded and given a learnable positional encoding before the encoder stack; the day-ahead exogenous forecasts are embedded in parallel; the two representations are concatenated and mapped to the n output periods by an MLP.

The price branch takes the previous $m{=}14$ days, each a vector of n period prices; each day is linearly embedded to 128 dimensions, a learnable positional encoding is added, and the sequence passes through six encoder layers (eight attention heads, feed-forward dimension 1024, dropout 0.2), the final position giving the price representation. The exogenous branch linearly embeds the day-ahead forecasts. The concatenated representations are mapped by a single-hidden-layer perceptron (width 128) to the n output periods.

Training uses Adam at learning rate 10^{-4}, batch size 32, up to 200 epochs with early stopping (patience 25) on an internal validation split, under a step learning-rate schedule. The number of exogenous inputs is fixed at two to match the paper, and the remaining hyperparameters follow the paper's reported architecture and search ranges rather than a per-market optimum: the 128-dimensional embedding lies within the paper's range but below its Nord Pool optimum of 512, and per-market tuning is left as future work (§3.7). Because this wide feature vector is interpreted positionally rather than by column name, it is produced by a dedicated feature pipeline (`default_transformers_for_transformer`) that lays out the $m \times n$ price block followed by the exogenous block in a fixed order.

3.4 Recalibration and backtesting

All three models are evaluated by a single walk-forward backtesting engine that reproduces the rolling-recalibration scheme of Lago et al. [3], illustrated in Fig. 3.4. For each delivery day in the test window, and for each calibration-window length in the model's ensemble, the engine slices the training data to the most recent window of days strictly preceding the target day, fits the feature and target scalers on that slice alone, trains the model, and predicts the target day; the day's forecast is the arithmetic mean of the predictions across the windows.



each test day repeats this; the window slides one day forward and models refit (daily: LEAR, DNN; weekly: Transformer)

Figure 3.4: Rolling walk-forward recalibration with a calibration-window ensemble, after Lago et al. [3]. For each test day the data are sliced to the most recent window of each ensemble length (shown for LEAR: 56/84/1092 days), every window is refit and predicts the target day, and the forecast is their average. The boundary then advances one day; models refit daily (LEAR, DNN) or weekly (Transformer).

To bound compute, each window keeps a cached fitted model and is refitted only when the days since its last fit reach the configured refit interval, daily for LEAR and the DNN and weekly for the Transformer, following the respective papers. LEAR uses the paper ensemble of 56-, 84-, and 1092-day windows; the 1456-day window of the original specification is dropped because it predates the available data. The DNN uses a single 730-day window, and the Transformer a single 1092-day window, verified to leave enough usable sequence rows before the test start in both markets (at least 1092, and 1265 measured for DE). The train/test split is date-based: training data are all rows strictly before the test start date, and no validation holdout is carved out of the backtest itself (the validation window of §3.6 is used only by the tuning command).

3.5 Evaluation protocol

The test window runs from 2025-10-24 to 2026-05-31 (220 calendar days) and is identical for all six runs. Four accuracy metrics are reported. For N aligned prediction–actual pairs

$(\hat{y}_i, y_i)$,

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}, \tag{3.4}$$

$$\text{sMAPE} = \frac{100}{N}\sum_{i=1}^{N}\frac{|y_i - \hat{y}_i|}{\frac{1}{2}(|y_i| + |\hat{y}_i|)}, \qquad \text{rMAE} = \frac{\text{MAE}(\mathbf{y}, \hat{\mathbf{y}})}{\text{MAE}(\mathbf{y}, \hat{\mathbf{y}}^{\text{naive}})}, \tag{3.5}$$

where the naive benchmark $\hat{\mathbf{y}}^{\text{naive}}$ is the weekly-naive forecast (the realised price exactly seven days earlier), as recommended by Lago et al. [3], §5.4. The rMAE is the headline metric because, being normalised by the same-market naive error, it is the figure that can be compared across periods and markets.

Statistical significance is assessed with the multivariate Giacomini–White test [8, 3]. For each ordered pair of models (A, B) the daily loss differential

$$d_t = \sum_{h=1}^{n}\left(|e_{t,h}^{A}| - |e_{t,h}^{B}|\right) \tag{3.6}$$

is formed over the n delivery periods of day t (the L^1 norm of the errors), and the test statistic $\bar{d}/\hat{\sigma}_{\bar{d}}$ is computed with a Newey–West heteroskedasticity- and autocorrelation-consistent standard error using a lag of $\lfloor T^{1/4} \rfloor$ over the T test days. The univariate Diebold–Mariano test [7] is reported as a complement. Models are compared on the test days that all of them produced, since the different feature pipelines drop gap and daylight-saving days slightly differently across model families. Finally, computation cost is reported as the training time per recalibration, read from the experiment tracker, as the analogue of Lago et al. [3], Table 4.

3.6 Hyperparameter tuning

Only the DNN is tuned; LEAR cross-validates λ internally and the Transformer reuses the paper optimum (Table 3.3). Its six architecture hyperparameters are searched with Optuna's Tree-structured Parzen Estimator over hidden-layer width (16–512, log), activation (relu, tanh, elu, softplus, or sigmoid), dropout (0–0.5), learning rate (10^{-5}–10^{-1}, log), the batch-normalisation flag, and the L1 penalty (10^{-8}–10^{-2}, log). The objective is the mean MAE over a strictly ex-ante validation window (2025-01-01 to 2025-10-23, the 42 weeks before the test window; [3], §4.3.2), scored under the same walk-forward engine as the backtest, with each trial persisted to `algo.hyperparam_trials` for resumable search. Lago's ten binary feature- and selector-toggles, and their `preprocessing` and `init_method` choices, are omitted: the preprocessor exposes no such switches, so they would only add inert search dimensions (Table 3.5).

3.7 Deviations from the source papers

The replication is faithful in specification but departs from the source papers in a small number of documented respects, summarised in Table 3.5. Each deviation is driven either by the length of the available data history (which begins on 2022-03-10) or by a compute budget, and none changes the model class or the feature definition.

Table 3.5: Documented deviations from [3] and [6].

Aspect	**Paper**	**This work (reason)**
Test length	$\geq$ 1–2 years	31 weeks (price history starts 2022-03, and validation and calibration consume the rest)
DNN tuning budget	1500 TPE iterations	30 trials (compute, with the optimum stable early in prior studies)
DNN search space	architecture + feature selectors	architecture only (feature set fixed to the paper specification)
LEAR windows	56 / 84 / 1092 / 1456 d	56 / 84 / 1092 d (1456 predates the data)
DNN window	4 years	730 d (data and compute constraint)
λ selection	LARS+AIC, then coordinate-descent refit	`LassoCV` time-series CV (standard, close accuracy)
Market resolution	hourly (paper era)	both markets now 15-min, downsampled to an hourly mean for cross-market and cross-paper comparability
Transformer pipeline	train-once + renormalise	calibration window + weekly refit (consistency with the other models)
Transformer HPs	tuned per dataset	paper architecture / search-range defaults reused (per-market tuning is future work)

4 System Implementation

4.1 Scope and approach

4.1.1 Scope of this chapter

This chapter documents the *system* built around the forecasting models. That system comprises the orchestration layer, infrastructure choices, observability stack, and CI/CD pipeline that together let the platform run reliably in production. It does not cover the modelling research itself, such as LEAR hyperparameters, DNN architecture, the asinh price transformation, or the Diebold–Mariano and Giacomini–White statistical tests (Chapter 3). The exception is where a modelling decision directly motivates or constrains an MLOps practice. Operational how-to material such as running backfills, adding new bidding areas, or re-deploying Prefect flows is kept in the repository's `RUNBOOK.md` and `CONTRIBUTING.md` and is not repeated here.

4.1.2 Structuring the chapter around Zarour's practices

The chapter is organised around the nine MLOps practices identified in the systematic literature review by Zarour et al. [4]. These are automating CI/CD, orchestrating workflows, ensuring reproducibility, versioning, collaboration and communication, continuously training and evaluating models, tracking and logging ML metadata, continuous monitoring, and implementing feedback loops. Each practice section follows a fixed shape. It gives Zarour's own framing of the practice and what it requires of a production ML system, the concrete implementation found in algo-repo, the technology chosen together with the alternatives that were considered and rejected, and the design rationale given the single-host MVP scope of the platform. This structure makes it straightforward to assess which practices are fully implemented, partially implemented, or deliberately deferred, and to trace every technology choice back to an explicit justification. The same nine-practice vocabulary structures the maturity assessment in Chapter 6.

4.2 System at a Glance

Before examining each Zarour practice in detail, this section presents the platform as a whole, covering its host-level topology, the end-to-end data plane, the complete inventory of scheduled deployments, and the technology choices that underpin every layer.

4.2.1 Host-level architecture

The platform runs entirely within a single Docker Compose stack on one dedicated host, keeping operational complexity proportionate to the MVP scope. Six long-lived services form the persistent backbone of that stack. ClickHouse stores all time-series data and ML artifacts. The Prefect server exposes the orchestration API and web UI. The Prefect worker polls for scheduled runs and manages a Docker work pool. Loki receives structured log streams from Python via `python-logging-loki`, and Prometheus scrapes service metrics from ClickHouse and Grafana. Grafana visualises the ClickHouse data, the Loki log stream, and the Prometheus series through file-provisioned dashboards. When the Prefect worker dispatches a scheduled flow run, it spawns an ephemeral container from the `algo-repo:latest` image. That container reads from and writes to ClickHouse, calls the ENTSO-E Transparency Platform API, and emits structured logs back to Loki before being removed automatically on completion.

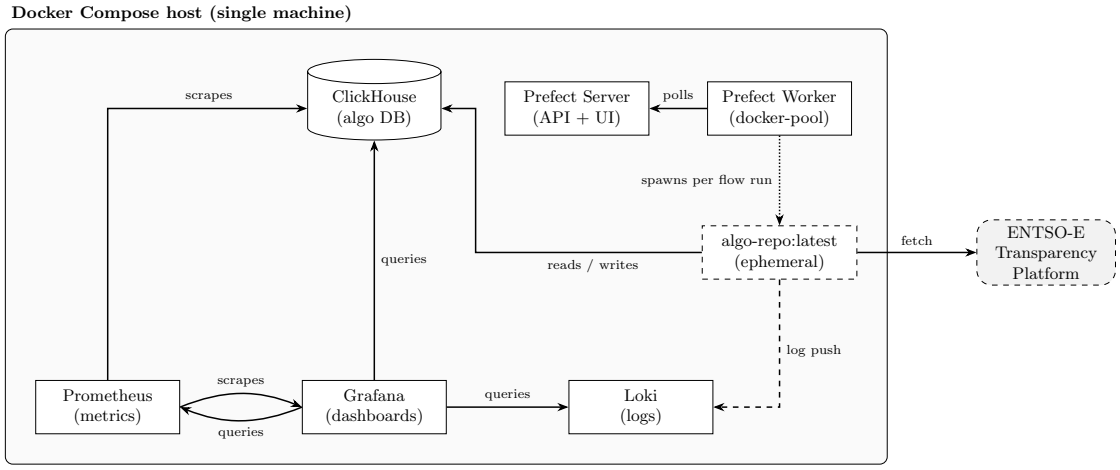


Figure 4.1: Host-level architecture. A single Docker Compose host runs the six long-lived services, namely ClickHouse, the Prefect server and worker, Prometheus, Grafana, and Loki. Flow runs spawn ephemeral `algo-repo:latest` containers that read and write ClickHouse, fetch from ENTSO-E, and push logs to Loki; Prometheus scrapes service metrics and Grafana queries Prometheus and Loki.

4.2.2 Data plane

Data enters the platform through the ENTSO-E Transparency Platform API, where four fetch functions retrieve energy prices, total-load forecasts, generation forecasts, and wind and solar forecasts for each configured bidding area. The raw XML responses are parsed by typed mapper functions that produce `Observations` and `Forecast` domain objects, which are then ingested into ClickHouse tables `algo.timeseries` and `algo.forecasts` via Arrow-batch inserts. Both tables use `ReplacingMergeTree` engines, which deduplicate lazily on merge. Read paths therefore either append the `FINAL` keyword to their queries or call `drop_duplicates` on the loaded DataFrame to ensure correctness. A preprocessor loads this data, pivots it to a daily-wide feature matrix through a configurable sequence of feature transformers, and feeds it to either the backtest engine for offline evaluation or the live runner for day-ahead production forecasting. Predictions and trained model artifacts are persisted to `algo.experiment_runs`, `algo.experiment_predictions`, and `algo.model_artifacts`, from where the reconciliation flow back-fills actuals the following day and the drift-detection flow compares the recent price distribution against a ninety-day reference window, triggering an alert and an optional automatic retraining run when significant drift is detected. Figure 4.4 draws this end-to-end path together; it is deferred to the end of the chapter (§4.4) so that each stage is introduced first.

4.2.3 Operational inventory

Table 4.1 lists every Prefect deployment currently registered in the system, together with its schedule, the flow it executes, and the Zarour practice it primarily realises.

Table 4.1: Operational inventory of Prefect deployments. Cron strings are UTC; the seven backfill deployments and six forecast deployments share one flow definition apiece.

Deployment	Area / model	Cron	Flow	Purpose	Zarour
entsoe-backfill-dk1	DK1	0 * * * *	`entsoe_backfill_flow`	Hourly ENTSO-E ingest	P2
entsoe-backfill-dk2	DK2	0 * * * *	*(as above)*	Hourly ENTSO-E ingest	P2
entsoe-backfill-nl	NL	0 * * * *	*(as above)*	Hourly ENTSO-E ingest	P2
entsoe-backfill-be	BE	0 * * * *	*(as above)*	Hourly ENTSO-E ingest	P2
entsoe-backfill-de	DE	0 * * * *	*(as above)*	Hourly ENTSO-E ingest	P2
entsoe-backfill-fr	FR	0 * * * *	*(as above)*	Hourly ENTSO-E ingest	P2
entsoe-backfill-pl	PL	0 * * * *	*(as above)*	Hourly ENTSO-E ingest	P2
daily-forecast-dk1_lear	DK1 / LEAR	0 6 * * *	`daily_forecast_flow`	Daily refit + forecast	P6
daily-forecast-de_lear	DE / LEAR	0 6 * * *	*(as above)*	Daily refit + forecast	P6
daily-forecast-dk1_dnn	DK1 / DNN	0 6 * * *	*(as above)*	Daily refit + forecast	P6
daily-forecast-de_dnn	DE / DNN	0 6 * * *	*(as above)*	Daily refit + forecast	P6
daily-forecast-dk1_transformer	DK1 / Transformer	0 6 * * *	*(as above)*	Daily refit + forecast	P6
daily-forecast-de_transformer	DE / Transformer	0 6 * * *	*(as above)*	Daily refit + forecast	P6
reconcile-predictions	all	0 18 * * *	`reconcile_predictions_flow`	Day-after reconciliation	P7, P8
drift-detection-dk1	DK1	0 19 * * *	`drift_detection_flow`	Drift: 90d ref vs 7d (Evidently)	P8, P9
drift-detection-de	DE	0 19 * * *	*(as above)*	Drift: 90d ref vs 7d (Evidently)	P8, P9

4.2.4 Technology stack

Table 4.2 lists each tool and its role; the rationale for each choice is given in the practice section where the tool appears.

Table 4.2: Technology in use and its role. The rationale for each choice is given in the practice section where the tool appears; tools evaluated but not adopted (MLflow, Kubernetes, DVC) are discussed there too.

Tool	Role
Prefect	Orchestration
ClickHouse	OLAP store + model-artifact blob
Docker Compose	Container runtime + infrastructure-as-code
Loki	Log aggregation
Prometheus	Metrics time-series store
Grafana	Dashboards and file-based alerts
Evidently AI	In-process drift detection
W&B	Research + production tracking
GitHub Actions	CI/CD
Tailscale	Deploy-host network access
Optuna	Hyperparameter search (TPE)

4.3 Implementing Zarour's Nine MLOps Practices

The nine sections that follow map every MLOps practice identified by Zarour et al. (Chapter 2) to a concrete subsystem in algo-repo, in Zarour's order from P1 to P9. Each follows the same shape, covering the practice's framing, the implementation found in this repository, the technology chosen and its rejected alternatives, and the design rationale scoped to a single-host MVP. Where a practice is only partially realised, the limitation is stated explicitly and cross-referenced to the maturity assessment in Chapter 6.

4.3.1 Automating CI/CD

Automating continuous integration and delivery requires that every proposed change be validated before it reaches `main`, and that an accepted change then propagate to the running system without manual intervention [4]. In algo-repo this is a two-phase GitHub Actions pipeline (Figure 4.2).

On a pull request, three checks must pass before a merge is allowed. `pr_test` runs the full pytest suite mirroring `src/`, `pr_ruff-lint` runs the `ruff` linter, and `pr_smoke` runs an end-to-end daily-forecast and reconciliation on a throwaway Compose stack with synthetic data. A path filter skips the expensive smoke build for documentation-only changes.

On merge to `main`, `cd_deploy` reaches the production host and, gated by which paths changed, applies any new ClickHouse migrations, rebuilds the `algo-repo:latest` image with the commit SHA baked in, and reruns `deploy_ingestion.py` and `deploy_forecast.py` to register the updated Prefect deployments. It finally triggers `daily-forecast-dk1_lear` and polls until it completes, confirming the deployed image produces a valid forecast on the live host. A parallel job restarts Grafana when its provisioning changes.

The deploy host is reached only over a Tailscale mesh. The runner joins the tailnet for the duration of the workflow with an ephemeral key, so the host never exposes an SSH port to the public internet. GitHub Actions was chosen over Jenkins or CircleCI because it is co-located with the repository and needs no separately managed runner.

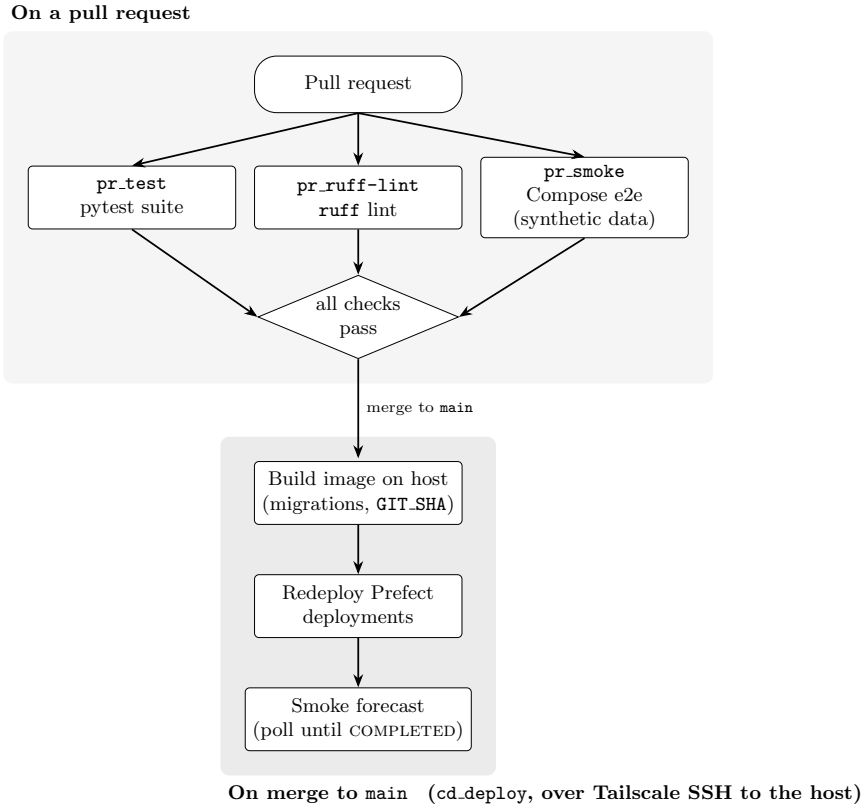


Figure 4.2: The CI/CD pipeline. On a pull request, three checks (`pr_test`, `pr_ruff-lint`, `pr_smoke`) must pass before a merge. On merge to `main`, `cd_deploy` reaches the host over Tailscale, rebuilds the image, redeploys the Prefect flows, and runs a smoke forecast.

4.3.2 Orchestrating workflows

Orchestrating workflows, in Zarour et al.'s framing, is the practice of expressing the recurring data and model tasks as scheduled, observable, retryable pipelines rather than as cron-driven scripts, so that failures are isolated, retried, and surfaced rather than silently lost [4].

All recurring work on the platform is expressed as four Prefect flows executed by a Docker work pool (Table 4.1). These are ingestion, daily forecast, reconciliation, and drift detection. Scheduling is declarative. Each deployment in Table 4.1 attaches a cron schedule to a flow, and the worker spawns an ephemeral `algo-repo:latest` container per run. The ingestion flow is parameterised by area, a single flow definition with one deployment per bidding zone, so adding a market is a one-line deployment change with no flow-code edit. Within a flow, granularity is one task per data type, covering prices, load, generation, and wind/solar. Tasks run sequentially to respect the ENTSO-E rate limit and each carries `retries=2` with a 60-second delay, so one data type failing does not abort the others. The ingestion flow re-ingests a rolling 48-hour lookback window on every run. Because both ClickHouse tables are `ReplacingMergeTree`, re-ingesting overlapping data is always safe.

A deliberate design choice keeps the fetch-and-insert logic in a shared ingest service with no Prefect dependency, so that both the data-backfill CLI and the ingestion flow import the same code path rather than duplicating it. Prefect was chosen over Airflow and Dagster. At single-host scale its Python-native flow definitions and Docker work pool are sufficient,

and it avoids the operational weight of an Airflow scheduler or a Dagster daemon and database.

4.3.3 Ensuring reproducibility

Reproducibility in the context of MLOps requires that a past model result can be reconstructed from the same code version, data window, hyperparameters, and *runtime environment*, without relying on implicit host state [4]. Zarour et al. identify it as one of the most frequently cited challenges in practitioner surveys, and treat containerisation of the runtime as one of its principal enablers.

The platform addresses reproducibility along three axes. First, the runtime is fully containerised and the topology is infrastructure-as-code. The entire operational stack, comprising all six long-lived services with their volumes, environment, and network, is declared in `docker-compose.yml`; the flow-run and worker containers are built from `Dockerfile` and `Dockerfile.worker`; and Grafana's provisioning, the alert rules, and the Prometheus scrape config are all file-mounted from the repository. The infrastructure can therefore be rebuilt from the repository alone, with no hand-applied host commands. Second, every row in `algo.experiment_runs` records its git SHA and the data window it used, which together fix the code version and the data behind any run. The `GIT_SHA` is baked into the image at build time rather than resolved at runtime, because the deploy host holds only the built image and not the source tree, and local runs emit an empty string so a dirty tree never stamps a misleading SHA. Third, the modelling pipeline is itself deterministic given the config. Calibration windows and the `AsinhScaler` target transform are fixed in the committed configs, and the DNN ensemble averages `n_dnn_ensemble` replicates with distinct, enumerated random seeds, so the trained ensemble is fully described by the config and a deterministic seed sequence.

Docker Compose was chosen over Kubernetes for the container runtime. At single-host scope, a control plane, etcd, and Helm charts are unjustified overhead (§4.4). Baking the SHA into the image rather than resolving it at container start makes every `algo.experiment_runs` row self-describing under a deployment model that has no source checkout on the host.

4.3.4 Versioning

Versioning is the foundational practice under which all code, data schemas, and configuration that affect model behaviour are tracked in source control, enabling reproducible reconstruction of any past system state [4]. Zarour et al. treat it as a prerequisite for every other practice, because without a shared history of code and configuration changes, claims about reproducibility or rollback cannot be substantiated.

The platform uses Git with GitHub as the remote, and versions every artefact that can affect a model run. Python experiment configs in `configs/` are first-class Git files whose history records every calibration-window or scaler change. ClickHouse schema changes live in numbered SQL migration files under `src/infrastructure_clickhouse/migrations/` that are auto-discovered in sorted order by `run_migrations.py` and never modified retroactively. Grafana dashboard definitions and alert rules, stored as JSON and YAML under `grafana/`, are file-provisioned, never edited through the UI, and subject to code review before taking effect.

DVC was evaluated for data versioning and rejected. ENTSO-E data is replayable in full from the public API, and combining ClickHouse's `ReplacingMergeTree` engine with the `git_sha` column on `algo.experiment_runs` provides sufficient provenance without maintaining a separate binary store. Versioning every artefact that affects model output means a single `git log` spans the experiment, the infrastructure, and the alerting policy

together, closing the gap between operational changes and the audit trail expected of a production ML system.

4.3.5 Collaboration and communication

Collaboration practices let contributors work on a shared ML system without relying on tribal knowledge [4]. As a single-developer project this was the least-emphasised of the nine practices, but the core disciplines were still followed. Every code change went through a GitHub *pull request* rather than a direct commit to `main`, so each change passed the CI gate of P1 (Figure 4.2) and left a reviewable record of what changed and why. GitHub *Issues* tracked outstanding work and the status of each feature branch, giving a single queue of what was in progress, blocked, or done.

Beyond the version-control workflow, project conventions, repository structure, and operational procedures are kept in versioned Markdown files, namely `CLAUDE.md`, `docs/REPO_OVERVIEW.md`, `docs/RUNBOOK.md`, and `docs/CONTRIBUTING.md`, so the design rationale and the onboarding path for a future contributor live in the repository itself rather than in any one person's head.

4.3.6 Continuously train and evaluate

Continuous training and evaluation requires that the model be retrained automatically on a regular cadence so that it reflects the most recent data distribution, and that its live predictions be continuously scored against realised outcomes [4]. Zarour et al. distinguish the scheduled, unconditional refit from the reactive drift-triggered path of P9.

The platform's train-and-evaluate cycle runs on a fixed daily cadence (Figure 4.3). The six `daily-forecast-*` deployments, covering LEAR, DNN, and Transformer for both DK1 and DE, execute the daily-forecast flow at 06:00 UTC. Each run invokes the live runner, which loads the latest ClickHouse data window, refits the ensemble of calibration-window models from scratch, and immediately produces a day-ahead forecast. For DNN configurations it fits `n_dnn_ensemble` seed replicates and averages them, following Lago et al. [3]. The trained models are serialised to `algo.model_artifacts`. The *evaluate* half of the practice is the reconciliation flow (§P7–P8). The day after each forecast, realised prices are joined to the stored predictions and the same MAE, RMSE, and sMAPE used in backtesting are computed on the live forecast, so offline and online evaluation use one metric implementation.

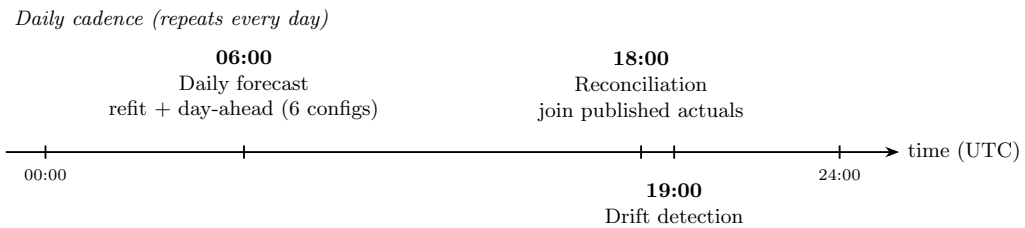


Figure 4.3: The daily train-and-evaluate cadence (UTC). At 06:00 every configuration refits from scratch and produces a day-ahead forecast; at 18:00 reconciliation joins the published actuals to the stored predictions and scores them; at 19:00 drift detection runs. The cycle repeats every day.

A full refit from scratch on each run was chosen over warm-start learning. The daily data increment is small relative to the calibration windows, so a clean refit is cheap and avoids accumulated approximation error. Keeping training and prediction in one flow means a

single task-level retry covers both phases and the served model is always consistent with the forecast it produced.

4.3.7 Track and log ML metadata

Tracking and logging ML metadata is the practice of recording the full provenance of every model run in a queryable store, so that past results can be reproduced and compared without ad-hoc file management [4]. That provenance covers the hyperparameters, data windows, performance metrics, and the model artefact itself.

This is handled in ClickHouse. `algo.experiment_runs` stores a self-describing row per run, holding its training duration, data window, git SHA, and a link to the stored model. `algo.experiment_predictions` holds one row per period per day for reconciliation, `algo.model_artifacts` stores the serialised models as blobs, and `algo.hyperparam_trials` persists every Optuna trial for resumable search. On the research surface, W&B hosts two projects, `algo-research` for backtests and sweeps and `algo-prod` for production reconciliation; `WANDB_MODE=disabled` no-ops it in CI and local development.

A filesystem model store was rejected. Storing artefacts as blobs in `algo.model_artifacts` keeps them attached to the experiment row rather than in a separately managed path. MLflow was evaluated and deferred until the experiment-config count exceeds ten. At the current scale ClickHouse and W&B together cover the audit-log and research-surface roles without adding a service to the Compose stack. Using one OLAP store for predictions, metrics, and artefacts makes day-after reconciliation a single query, and the `ReplacingMergeTree(created_utc)` engine makes the reconciliation update a safe re-insert rather than a mutable `UPDATE`.

4.3.8 Continuous monitoring

Continuous monitoring instruments the platform across Zarour's three axes, namely data quality, model performance, and system health, so that degradation surfaces automatically rather than by manual inspection [4]. Table 4.3 maps each axis to the surface that presents it and the store it reads. Structured logs from every Prefect flow and infrastructure module are pushed straight to Loki through a `python-logging-loki` handler, and Grafana auto-provisions three dashboards over ClickHouse, Prometheus, and Loki from `grafana/dashboards/`. The division of surfaces is deliberate. Grafana owns platform-and-data-health and W&B owns model quality, and both read from the same ClickHouse tables.

Table 4.3: Monitoring coverage. The first three rows are Zarour's P8 axes; *Operational* is the shared structured-log stream beneath them. Grafana and W&B both read from ClickHouse.

Axis / role	Surface	Source	What it watches
Data quality	Grafana *Data Quality*	ClickHouse	Freshness, daily row counts, and missing price hours in `algo.timeseries` and `algo.forecasts`
System health	Grafana *System Health*	Prometheus	Service uptime, scrape status, query rate, memory, and merge backlog
Model performance	W&B `algo-prod`	ClickHouse	MAE, RMSE, and sMAPE per reconciled day, area, and config
Operational logs	Grafana *Operational*	Loki	Log stream by `job=algo-repo`, plus error rate and logs by service

Five conditions raise an alert, listed in Table 4.4.

Table 4.4: Alert rules and their severities. The first four are file-provisioned Grafana rules in `grafana/provisioning/alerting/rules.yaml`, evaluated over ClickHouse except `clickhouse-down`, which reads the Prometheus liveness metric. `accuracy-degradation` is a Slack check inside the reconciliation flow.

Alert	Fires when	Severity
`data-staleness`	No fresh price row in twelve hours	warning
`drift-detected-today`	A `drift_reports` row is flagged today	warning
`reconciliation-missed`	No reconciled run in the last 36 hours	error
`clickhouse-down`	`up{job="clickhouse"}` is zero for five minutes	critical
`accuracy-degradation`	Today's MAE exceeds $1.5\times$ the 30-day median	warning

Loki was chosen over the ELK stack as a single binary with far lower memory and operational overhead, proportionate to a single host. Prometheus sits alongside Loki as the metrics tier to its logs tier, scraping ClickHouse, Grafana, and itself every 15 seconds and retaining 120 days of series, with Grafana the single dashboard and alerting surface above both.

4.3.9 Implementing feedback loops

The feedback-loop practice asks that a system detect when its inputs or outputs have shifted enough to degrade quality and respond automatically, rather than waiting for a human to notice [4]. This is the least developed of the nine practices on the platform and one of its weaker points. Drift detection itself runs daily and in-process, inside the `drift-detection-dk1` and `drift-detection-de` deployments, via Evidently's `DataDriftPreset` comparing a 90-day reference window against a 7-day current window on each area's price series. Every result is written to `algo.drift_reports`, and a detected event posts a warning to Loki and raises an alert.

The automatic response, however, is wired but inactive. A drift event triggers a retraining run only when a forecast config sets `retrain_on_drift=True`, which none of the six production configs does, so the flag stays false everywhere and drift in practice serves as a monitoring and alerting signal rather than a closed loop. The retrain path is nonetheless implemented and auditable, namely a triggered flow that records an audit row in `algo.retrain_events` and links the resulting run to the `drift_report_id` that caused it. Whether closing the loop is worthwhile at all, given the cheap daily refit that already retrains every model from scratch, is examined in Chapter 6.

4.4 Reproducibility, monitoring, and single-host scope

With every practice now described, Figure 4.4 draws them together into the single data path the platform implements, from ingestion into the store, through the forecasting pipeline that reads it, to the daily reconciliation-and-drift loop that closes back onto the store.

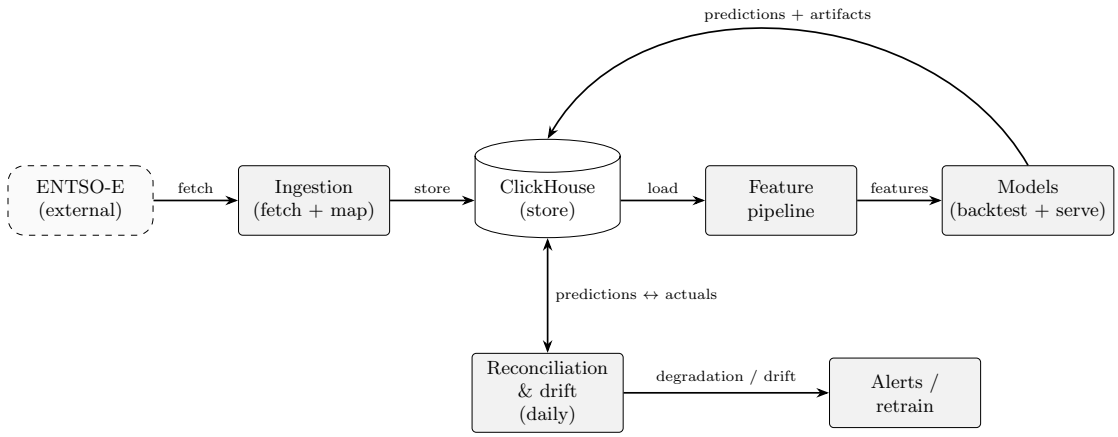


Figure 4.4: Data plane (high level). ENTSO-E data is ingested into ClickHouse; the forecasting pipeline loads it, builds features, and trains and serves the models, writing predictions and artifacts back. The daily reconciliation-and-drift loop reads those predictions, writes back actuals and metrics, and raises alerts or retraining.

4.4.1 GIT_SHA propagation as the reproducibility–monitoring link

Every row that `cd_deploy.yml` produces in `algo.experiment_runs` carries the same `git_sha` baked into the image during the deploy-gate build step, the SHA of the exact commit merged to `main`. The Grafana Operational dashboard queries the Loki log stream filtered by `job=algo-repo` and presents structured events that include the same SHA, because every flow container is built from that image. So if the W&B `algo-prod` run for an `(area, config_name)` shows an unexpected MAE spike beginning on a particular day, the Operational dashboard shows the Loki events from the same period alongside the `git_sha` of the image that produced them, making it possible to `git log` back to the exact commit. This links the reproducibility of P3 to the monitoring of P8. The audit trail for a production anomaly runs from the Grafana panel, through the `algo.experiment_runs` row, to a checkable commit, with no gap.

4.4.2 Why a single-host deploy is the right MVP scope

The entire stack runs on one Docker Compose host, for two reasons. First, Kubernetes and multi-host orchestration were out of scope for this project. The operational complexity of a control plane, distributed storage for ClickHouse, and cross-host log aggregation

would have slowed iteration without improving the platform's ability to deliver daily forecasts. Such an architecture would, however, genuinely improve robustness, since it removes the single failure domain and enables rolling deployments and horizontal scaling. It is the natural next step once a second host is justified. Second, only one compute instance was available, so a distributed deployment was not an option in any case. On a single host the blast radius is one failure domain, and rollback is simply rebuilding the image and restarting the stack.

5 Results: Forecasting Models

This chapter reports the head-to-head comparison of the three forecasting models specified in Chapter 3. Six backtests were run, covering LEAR, the DNN, and the Transformer, each on DK1 and DE, over a single shared hourly test window (2025-10-24 to 2026-05-31) with the paper-exact feature sets. Within each market the models are compared on the test days that all of them produced. Because the Transformer's fourteen-day input sequence consumes more leading days than the other pipelines, this common set is 187 days on DK1 and 176 on DE. The chapter reports headline accuracy (§5.1), statistical significance (§5.2), and computation cost (§5.3), then sets the results against the published benchmarks (§5.4).

5.1 Headline accuracy

Table 5.1: DK1: final benchmark metrics (test 2025-10-24–2026-05-31, common days).

	MAE	RMSE	sMAPE	rMAE
LEAR	17.574	26.891	29.048	0.469
DNN	17.960	25.634	29.339	0.482
Transformer	20.076	28.909	31.767	0.543
Naive (weekly)	36.899	51.385	50.110	1.000

Table 5.2: DE: final benchmark metrics (test 2025-10-24–2026-05-31, common days).

	MAE	RMSE	sMAPE	rMAE
LEAR	13.069	25.041	22.107	0.376
DNN	13.476	22.544	22.763	0.388
Transformer	14.719	23.755	23.901	0.420
Naive (weekly)	35.049	51.914	46.090	1.000

Tables 5.1 and 5.2 report the four accuracy metrics on the common test days. The ordering is the same in both markets and across MAE, rMAE, and sMAPE. LEAR is the most accurate, the DNN follows closely, and the Transformer is third, with all three improving substantially on the weekly-naive baseline. The rMAE values are 0.469 (LEAR), 0.482 (DNN), and 0.543 (Transformer) on DK1, and 0.376, 0.388, and 0.420 respectively on DE. Because rMAE normalises each model against the same-market naive error, these figures are comparable across markets. All three models sit markedly further below 1 on DE than on DK1, indicating that the German price was the more predictable of the two relative to its own naive benchmark over this window.

The single metric that reorders the models is RMSE. The DNN attains the lowest RMSE in both markets (25.6 against LEAR's 26.9 on DK1, and 22.5 against 25.0 on DE), and on DE LEAR in fact posts the highest RMSE of the three, behind even the Transformer. As RMSE weights large deviations more heavily than MAE, this is evidence that the DNN suppresses the largest forecast errors more effectively than LEAR even on the days where

LEAR's mean error is slightly lower. LEAR's advantage is thus specific to the absolute-error metrics that Lago et al. [3], §5.4, designate as primary. On the squared-error criterion the DNN is preferable. Whether either difference is statistically meaningful is the subject of §5.2.

5.2 Statistical significance

Table 5.3: DK1: GW multivariate p-values (column model better than row model).

	LEAR	DNN	Transformer	Naive (weekly)
LEAR	–	0.836	1.000	1.000
DNN	0.164	–	1.000	1.000
Transformer	0.001	0.001	–	1.000
Naive (weekly)	0.000	0.000	0.000	–

Table 5.4: DE: GW multivariate p-values (column model better than row model).

	LEAR	DNN	Transformer	Naive (weekly)
LEAR	–	0.807	0.988	1.000
DNN	0.193	–	0.994	1.000
Transformer	0.012	0.006	–	1.000
Naive (weekly)	0.000	0.000	0.000	–

The Giacomini–White multivariate test [8, 3] establishes whether these differences are significant. Tables 5.3 and 5.4 give the pairwise p-values, where cell (r, c) is the probability of a loss differential at least as large as the one observed under the null that column model c is no more accurate than row model r (the convention of [3], Fig. 5). A value below 0.05 means that c significantly outperforms r.

The central result is that LEAR and the DNN are statistically indistinguishable on both markets. The hypothesis that LEAR is no more accurate than the DNN cannot be rejected (DK1 $p = 0.16$, DE $p = 0.19$), and neither can its converse. LEAR's numerical lead in the headline tables therefore does not amount to a significant accuracy advantage, and the two models should be treated as equivalent in accuracy. Both, however, significantly outperform the Transformer (on DK1 at $p = 0.001$ for each, and on DE at $p = 0.012$ for LEAR and $p = 0.006$ for the DNN), and all three beat the weekly-naive baseline with $p \approx 0$. The comparison that survives significance testing is thus a two-way tie between LEAR and the DNN at the top, a significantly weaker Transformer, and a baseline that every model clears by a wide and significant margin, consistently across DK1 and DE.

5.3 Computation cost

Table 5.5 reports the amortised recalibration cost, the full-backtest wall-clock divided by the number of common test days, for each model and market. The figures invert the cost ordering of Lago et al. [3], Table 4, where on hourly CPU hardware LEAR trains in seconds and the DNN in minutes. Here the relationship is reversed. The GPU-trained MIMO networks recalibrate in one to two seconds per day, whereas LEAR (which fits an independent cross-validated Lasso for each of the 24 delivery periods across three

Table 5.5: Amortised recalibration cost (full-backtest wall-clock / common test day). Full timings in `Report/metrics/run_timings.md`.

Model	DK1 s/day	DE s/day	Hardware / cadence
LEAR	50.3	41.3	CPU, one LassoCV per period $\times$ 3 windows, daily
DNN	0.9	1.6	GPU, one MIMO net (single seed), daily
Transformer	1.6	1.9	GPU, one MIMO net, weekly refit

calibration windows, entirely on CPU) takes forty to fifty. The DNN is therefore roughly 25 to 55 times cheaper per recalibration than LEAR while, as §5.2 showed, being its statistical equal in accuracy. This inversion is a property of the hardware rather than of the models or the markets, since a GPU rewards the dense neural networks and penalises LEAR's many small per-period fits. On this platform it is the decisive practical difference between two otherwise-tied models. The DNN buys equal accuracy at around a fortieth of the compute. The Transformer, refitted weekly, carries a comparably low amortised cost but does not convert it into competitive accuracy.

5.4 Comparison with published benchmarks

A direct comparison of error levels with the source papers is complicated by the shift in price regime. The benchmarks of Lago et al. [3] were struck on 2016–2018 data, whereas this study covers 2025–2026, so the comparison is read through model rankings rather than absolute errors ([3], §5.4).

The rankings invert both source papers. Lago et al. [3] report the DNN as at least the equal of LEAR, and Llorente and Portela [6] report their Transformer ahead of the DNN. The ordering here is the reverse on both counts, LEAR $\geq$ DNN $\geq$ Transformer. As §5.2 showed, the LEAR–DNN inversion is only numerical, since the two are statistically tied, which is itself consistent with Lago et al.'s observation that the gap is often insignificant. The Transformer's third place, by contrast, is genuine and significant. The likeliest cause is that the deep models are under-optimised relative to LEAR in this study rather than inherently weaker. The DNN was given 30 tuning trials against the paper's 1500, and the Transformer reused the paper's Nord Pool hyperparameters without per-market retuning and was adapted into the rolling-refit pipeline rather than run in its native train-once-and-renormalise mode (Chapter 3, Table 3.5), while LEAR re-selects its penalty by cross-validation at every recalibration and so sits near its achievable best.

6 Results: System Evaluation

This chapter evaluates the system. The forecast-accuracy results are in Chapter 5. Following the single-case study guidelines of Runeson and Höst [11], the system is evaluated from its source code, version-control and CI history, the ClickHouse experiment tracker, and short targeted experiments. The figures are indicative of the system over the study window rather than claims about long-run production operation, a framing that matches comparable MLOps evaluations in the electricity-forecasting domain [5]. No controlled comparison against a second platform is attempted.

The evaluation composes four established instruments introduced in §2.5, three objective measurements and one structured self-assessment. An objective measurement is a number a script recomputes on demand from code, git history, or the database, via `scripts/report/extract_metrics.py` and `reproduce_run.py`. The self-assessment is the ML Test Score, a rubric whose every item carries one line of evidence a reader can check or contest. Between them the four instruments evidence eight of Zarour et al.'s nine practices [4]. The ninth, collaboration and communication, is qualitative and is documented in the system chapter (Chapter 4) rather than measured here. Each instrument is applied below and its result reported.

6.1 ML Test Score

The ML Test Score rubric of Breck et al. [12] scores 28 tests across four sections on a $\{0, 0.5, 1\}$ scale, the final score being the minimum of the four section sums so that the weakest area bounds the result. Table 6.1 reports the per-section sums. The full per-item evidence is in Appendix B.

Table 6.1: ML Test Score self-assessment (min-of-sections rule). Per-item evidence in Appendix B.

Section (max 7)	**Sum**
Data tests	2.5
Model development tests	3.0
ML infrastructure tests	4.0
Monitoring tests	4.0
ML Test Score (min)	**2.5**

The final score is **2.5**, bound by the data-testing section. On Breck et al.'s interpretation scale this falls in the $(2, 3]$ band, a system that is reasonably tested but could automate more of its tests and procedures. Within the data section the system scores for enforcing strict input schemas (`enforce_schema`) and for a tested feature-transformer pipeline, but does not formally document the feature-benefit, feature-cost, or privacy analyses the rubric also rewards, and has no data-quality CI gate. We read this as the system's primary remaining investment area. The score is deliberately conservative. Two data items could each be argued to 0.5 (that the Lasso performs feature selection, and that the feature set is grounded in Lago et al. [3]), which would lift the data section to 3.0 and the headline

to 3.0. We leave them at zero absent a documented analysis, so the reported score is a lower bound.

6.2 Reproducibility

Reproducibility is reported in two senses, namely provenance coverage across all runs and a worked re-run of one. The framing follows Gundersen and Kjensmo [13].

Table 6.2: Provenance triple coverage across 15 live runs in the window.

Provenance component	**Coverage**
`git_sha`	66.7 % (10 / 15)
`data_start_utc`	100.0 % (15 / 15)
`data_end_utc`	100.0 % (15 / 15)
`model_artifact_id`	100.0 % (15 / 15)
Full triple (all four)	**66.7 % (10 / 15)**

Of the 15 live runs in the window, 66.7 % carry the complete provenance triple (Table 6.2). The shortfall is entirely on `git_sha`: the data-window and artefact columns are populated for every run from day one, while five early runs predate the CD pipeline that bakes `GIT_SHA` into the production image. From the first cd-deployed run onward, all four components are present.

Table 6.3: Reproducibility re-run: regenerated from logged provenance, compared against the stored prediction.

Field	**Value**
Config	`dk1_lear`
Target date	2026-03-22
Delivery periods	24
Max absolute difference	7×10^{-6} EUR/MWh
Verdict	numerically identical
Command	`uv run python -m scripts.report.reproduce_run --config dk1_lear --date 2026-03-22`

For the demonstration (Table 6.3), the only inputs supplied were the run's committed config and target date, and the stored prediction was loaded from `algo.experiment_predictions`. For the deterministic LEAR configuration on 2026-03-22, the maximum absolute difference across all 24 delivery periods was 7×10^{-6} EUR/MWh, five orders of magnitude below any economically meaningful price unit, i.e. numerically identical up to solver-level floating-point tolerance. LEAR is used for the headline demonstration because it is deterministic given a fixed data window. The DNN and Transformer carry seed and ensemble stochasticity whose controlled re-run is left to future work.

6.3 Drift detection as monitoring, not a retraining trigger

The system refits every model from scratch daily on a rolling window at a cost of seconds (Chapter 4, P6), so it is never stale between refits and drift-*triggered* retraining would add no freshness benefit, following the cost-aware argument of Mahadevan and Mathioudakis [14]. Drift detection is therefore kept as a *monitoring and alerting* signal rather than a retraining trigger, and `retrain_on_drift` defaults to false. Over the window the detector flagged drift on 49 of 49 reports, an uninformatively high rate that calls for recalibration.

6.4 Maturity placement

The platform is positioned against the Google three-level maturity model [10]. Of the five models reviewed by Zarour et al. [4], Google is chosen because its three levels match the granularity of a single-host MVP, because its top level of CI/CD-pipeline automation is plausibly attainable where Microsoft's Level 4 of full automated operations would require multi-environment claims the platform does not make, and because it is the most-cited maturity reference in that review, giving the assignment an external touchpoint. Table 6.4 assigns a status and a Google level to each of Zarour's nine practices, with one line of evidence apiece. The three status labels map to the evidence already presented. A practice is *Realised* when a concrete, production-running artefact closes it end-to-end, and *Minimal* when the mechanism exists but its scope is intentionally restricted. Eight of the nine are Realised. The ninth, the feedback loop, is Minimal, since drift-triggered retraining exists but is scoped to two areas behind an opt-in flag, consistent with the monitoring-not-trigger argument of §6.3. The levels themselves are descriptive ordinals and support positioning rather than measurement. The quantitative load is carried by the ML Test Score and reproducibility results above.

Table 6.4: Maturity assessment of Zarour's nine practices, with the Google MLOps maturity model level [10] assigned to each.

Practice	Status	Google level	One-line evidence
P1 Automating CI/CD	Realised	2	`pr_test`/`pr_ruff-lint`/`pr_smoke` PR gates, `cd_deploy` on every merge to `main`, 20 production deploys over the window at about 1.56 per week
P2 Orchestrating workflows	Realised	2	Prefect flows with per-area deployments on a Docker work pool
P3 Ensuring reproducibility	Realised	2	`GIT_SHA` + data-window columns per run, containerised runtime in `docker-compose.yml`
P4 Versioning	Realised	2	Git: `configs/`, numbered migrations, dashboards and alert rules all version-controlled
P5 Collaboration & communication	Realised	1	`CLAUDE.md`, `REPO_OVERVIEW.md`, `RUNBOOK.md`, `CONTRIBUTING.md`, plus Slack alerts
P6 Continuously train & evaluate	Realised	1	Six `daily-forecast` deployments refit at 06:00 UTC, reconciliation scores live forecasts
P7 Track & log ML metadata	Realised	1	`algo.experiment_runs` + predictions + artifacts, plus the W&B research surface
P8 Continuous monitoring	Realised	2	Loki plus three Grafana dashboards (Data Quality, Operational, System Health) and four file-based alert rules, with model metrics in W&B `algo-prod`
P9 Implementing feedback loops	Minimal	1	Drift-triggered retrain behind a config flag, two areas only

7 Discussion

This chapter answers the three research questions of §1 and then draws the forecasting and system threads together. The forecasting answer (RQ1) rests on results reported in Chapter 5. The engineering and maturity answers (RQ2, RQ3) synthesise the system implementation and its evaluation.

7.1 RQ1: Comparing the forecasting models

The head-to-head comparison of LEAR, the DNN, and the Transformer across DK1 and DE is reported in Chapter 5, on a single shared test window with paired Diebold–Mariano and Giacomini–White tests, and against both a weekly-naive baseline and the rankings of Lago et al. [3] and Llorente and Portela [6]. Because the source papers' test periods predate the present price regime, the comparison is read through rMAE and model rankings rather than absolute errors.

Across both bidding zones the three models fall into two tiers. LEAR records the lowest mean absolute error, and with it the lowest rMAE and sMAPE, on both DK1 and DE (rMAE 0.469 and 0.376). The DNN follows closely (0.482 and 0.388), and the Transformer is third (0.543 and 0.420), with all three improving substantially on the weekly-naive baseline. The Giacomini–White test, however, cannot distinguish LEAR from the DNN on either market. The hypothesis that LEAR is no more accurate than the DNN gives $p = 0.16$ on DK1 and $p = 0.19$ on DE, and neither can its converse be rejected. LEAR's numerical lead is therefore not a significant accuracy advantage, and the two models are best regarded as equivalent. The same test confirms that LEAR and the DNN each significantly outperform the Transformer ($p \leq 0.012$) and that every model beats the naive forecast ($p \approx 0$). The answer to RQ1 is thus a statistical tie between LEAR and the DNN at the top, a significantly weaker Transformer, and a wide, significant margin over the naive benchmark, a pattern that holds identically on DK1 and DE.

Two refinements qualify this headline. First, the ranking depends on the error metric: although LEAR leads on the absolute-error measures, the DNN achieves the lowest RMSE in both markets, and on DE LEAR posts the highest RMSE of the three models. Because RMSE penalises large errors more heavily than MAE, the DNN evidently controls the worst misforecasts better than LEAR even where its mean error is marginally larger, a distinction that matters whenever large individual errors are disproportionately costly. Second, the accuracy tie does not extend to cost. On the platform's GPU the DNN recalibrates in about a second per day against LEAR's forty to fifty (Table 5.5), so it delivers statistically equal accuracy at a fraction of the compute. Where two models are tied on accuracy, this asymmetry becomes the deciding practical factor, and it favours the DNN.

Measured against the source papers, the comparison replicates one result and inverts two. It reproduces the finding of Lago et al. [3] that the accuracy gap between LEAR and the DNN is frequently insignificant. Here the two are statistically tied. It does not reproduce the papers' point rankings. Lago et al. place the DNN at least level with LEAR, yet here LEAR is numerically ahead (though tied), and Llorente and Portela [6] place their Transformer above the DNN, yet here the Transformer is significantly the weakest model. The most credible explanation is procedural rather than architectural. The deep models were optimised far less than LEAR in this study: the DNN was tuned for 30 TPE trials against the paper's 1500, and the Transformer reused the paper's Nord Pool hyperparameters

without per-market retuning, adapted into the rolling calibration-window-and-weekly-refit pipeline of the other models rather than its native train-once-and-renormalise scheme (Chapter 3, Table 3.5). LEAR, by contrast, re-selects its penalty by cross-validation at every daily recalibration and so sits close to its achievable best. The inversion is therefore better read as a measure of how much tuning the deep models need to express their reported advantage than as evidence that their architectures are weaker. Narrowing that gap is the leading item of modelling future work (Chapter 8).

7.2 RQ2: Engineering trade-offs of productionising an EPF benchmark

Embedding the benchmark models in a scheduled, containerised pipeline surfaced a series of trade-offs, each resolved in favour of the single-host MVP scope. The most consequential is the choice of a full refit from scratch on every daily run over warm-start or incremental learning: because the daily data increment is small relative to the calibration windows, a clean refit is cheap and avoids accumulated approximation error, and, as RQ3 and the drift discussion show, it also makes the model fresh enough that drift-triggered retraining becomes redundant. Co-locating training and prediction in a single Prefect flow trades a small amount of recomputation for the guarantee that the served model always matches the forecast it produced, removing a class of coordination failure. Using ClickHouse as the single store for time-series, predictions, metrics, and model artifacts makes day-after reconciliation one query rather than a cross-system join, at the cost of a base64-blob artifact representation that will not scale to large neural weights. And serving the paper-strict feature set in production, rather than a separate, richer operational feature set, keeps the deployed models bit-faithful to the benchmark, at the cost of forgoing features that might help operationally but would break the replication. In most cases the needs of a faithful benchmark and the needs of a small production platform lined up. Both favour simplicity, determinism, and a single source of truth.

7.3 RQ3: Critical practices and the limits of a single-host platform

Of Zarour's nine practices, eight are realised end-to-end and one, the feedback loop, is minimally realised (Table 6.4). The quantitative evaluation locates the binding constraint precisely. The ML Test Score of 2.5 is bound by the data-testing section, not by infrastructure or monitoring, which both score higher. In other words, for this platform the practices that proved easiest to realise to a high level were the DevOps-inherited ones (CI/CD automation, versioning, containerised reproducibility, and monitoring), because they transfer directly from software engineering, whereas the ML-specific data-quality discipline (feature-benefit, feature-cost, and slice analyses) is where the platform is weakest and where further investment would most raise its maturity. The single-host scope reaches its limit at three identifiable points. The first is the artifact store, which suits LEAR's hundred-kilobyte blobs but not multi-megabyte neural weights. The second is the absence of a model-quality gate before serving. The third is the coverage gap, with forecasting and drift scoped to two of the seven ingested bidding zones. None of these is a design flaw. Each is a deliberate boundary of the MVP with a named adoption trigger (Chapter 4).

7.4 Tying the threads together

The two threads are mutually reinforcing rather than independent. Several modelling decisions are only defensible because of the system around them: downsampling both markets to an hourly grid is what makes the cross-market and cross-paper comparison

valid, and the cheap, scheduled daily refit is what licenses treating drift detection as monitoring rather than as a retraining trigger, following the cost-aware argument of Mahadevan and Mathioudakis [14]. Conversely, the system's central claim, that any forecast can be reconstructed from its logged provenance, is what makes the forecasting replication itself trustworthy. The worked re-run that reproduced a logged LEAR forecast to within 7×10^{-6} EUR/MWh (§6.2) is evidence about the modelling thread as much as the system thread. The forecasting work tests the platform, and the platform makes the forecasting reproducible. Neither half of the thesis would stand on its own.

7.5 Limitations

The limitations span both threads and are collected here. On the modelling side, the test window is 31 weeks rather than the one-to-two years the source papers recommend, a consequence of the data history. The DNN hyperparameters are tuned on an architecture-only search space, with Lago's ten binary feature-selector hyperparameters left as fixed config decisions because the preprocessor exposes no toggles for them. The Transformer reuses the paper's search-range defaults rather than per-market optima, and each deep model is evaluated from a single seeded backtest run, so its result carries the variance of one draw, a caveat that bears most on the Transformer's third place. The work forecasts point prices, not probabilistic distributions. On the system side, the evaluation is a single-case study with no controlled comparison, the ML Test Score is self-administered and mitigated by the per-item evidence in Appendix B, forecasting and drift cover only DK1 and DE, the ClickHouse blob artifact store will not scale to large neural weights, and the drift detector over-flags. These are revisited as concrete future work in Chapter 8.

8 Conclusion and Future Work

This thesis took the benchmark electricity-price-forecasting models from the research literature and embedded them in an end-to-end production platform built along MLOps principles, then evaluated the result on two axes, the forecast quality of the models and the operational maturity of the system serving them.

On the system axis, eight of Zarour et al.'s nine MLOps practices [4] are realised end-to-end in the platform, and one, the feedback loop, is minimally realised, scoped to two bidding zones behind an opt-in flag. Every realised practice is traced to a concrete repository artefact, namely a Prefect deployment, a ClickHouse migration, a GitHub Actions workflow, or a Grafana dashboard. The qualitative "realised" labels are backed by measured values, each recomputable on demand from the live system. They comprise an ML Test Score of 2.5 bound by data testing, and provenance-triple coverage of 66.7 % supported by a re-run that reproduced a logged forecast to within 7×10^{-6} EUR/MWh. On the forecasting axis, the faithful replication of LEAR, the DNN, and the Transformer across DK1 and DE, evaluated with paired significance tests on a shared recent window, is reported in Chapter 5. On both markets LEAR proved the most accurate model and the DNN its statistical equal, indistinguishable under the Giacomini–White test ($p > 0.16$), while both significantly outperformed the pure Transformer and all three beat the weekly-naive baseline. This ordering inverts the rankings of the source papers, in which the DNN edges LEAR [3] and the Transformer leads the DNN [6]. The inversion is attributable to the deep models' reduced tuning budget in this study rather than to any inherent weakness of their architectures. Together, the two threads constitute a production ML platform that ingests live market data, recalibrates and forecasts daily, monitors prediction quality and data drift, and maintains a complete audit trail from every forecast back to the exact code commit and data window that produced it.

8.1 Future work

Several extensions follow directly from the limitations discussed in Chapter 7.

- **Modelling fidelity.** Restore the 1456-day calibration window and the LARS+AIC λ-selection of the original benchmark once the data history is long enough. Tune the Transformer per market rather than reusing the paper defaults, and run it in its native train-once-and-renormalise mode rather than the rolling refit used here. Run an `epftoolbox` side-by-side on the original 2016–2018 datasets to separate replication error from regime change.
- **Calibration-window sensitivity.** Use the `forecast-cli ablate` command to quantify how accuracy varies with calibration-window length, an ablation the present fixed-window matrix does not cover.
- **Probabilistic forecasting.** Extend the point forecasts to predictive distributions and evaluate with pinball loss or CRPS, the direction surveyed by Nowotarski and Weron [2].
- **Reproducibility of stochastic models.** Demonstrate the re-run for the DNN and Transformer under controlled seeds with GPU non-determinism suppressed, extending the deterministic-LEAR demonstration.

- **Data-testing maturity.** Formalise the feature-benefit, feature-cost, and privacy analyses and add a data-quality CI gate, the binding constraint on the ML Test Score.
- **Drift recalibration and coverage.** Recalibrate the drift detector, which currently flags every report, and extend forecasting and drift to the remaining ingested bidding zones.
- **Tooling adoption triggers.** Adopt MLflow, Kubernetes, a cloud-managed model store, or DVC if and when their named, observable triggers are reached (Chapter 4).

Bibliography

[1] Rafał Weron. “Electricity price forecasting: A review of the state-of-the-art with a look into the future”. In: *International Journal of Forecasting* 30.4 (2014), pp. 1030–1081. DOI: 10.1016/j.ijforecast.2014.08.008.

[2] Jakub Nowotarski and Rafał Weron. “Recent advances in electricity price forecasting: A review of probabilistic forecasting”. In: *Renewable and Sustainable Energy Reviews* 81 (2018), pp. 1548–1568. DOI: 10.1016/j.rser.2017.05.234.

[3] Jesus Lago et al. “Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark”. In: *Applied Energy* 293 (2021), p. 116983. DOI: 10.1016/j.apenergy.2021.116983.

[4] Mohammad Zarour, Hussein Alzabut, and Khalid T. Al-Sarayreh. “MLOps best practices, challenges and maturity models: A systematic literature review”. In: *Information and Software Technology* 183 (2025), p. 107733. DOI: 10.1016/j.infsof.2025.107733.

[5] Rakshith Subramanya, Seppo Sierla, and Valeriy Vyatkin. “From DevOps to MLOps: Overview and Application to Electricity Market Forecasting”. In: *Applied Sciences* 12.19 (2022), p. 9851. DOI: 10.3390/app12199851.

[6] Jesús Llorente and José Portela. “A Transformer Approach for Electricity Price Forecasting”. In: *arXiv preprint arXiv:2403.16108* (2025).

[7] Francis X. Diebold and Roberto S. Mariano. “Comparing Predictive Accuracy”. In: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263.

[8] Raffaella Giacomini and Halbert White. “Tests of Conditional Predictive Ability”. In: *Econometrica* 74.6 (2006), pp. 1545–1578.

[9] Bartosz Uniejewski, Rafał Weron, and Florian Ziel. “Variance Stabilizing Transformations for Electricity Spot Price Forecasting”. In: *IEEE Transactions on Power Systems* 33.2 (2018), pp. 2219–2229. DOI: 10.1109/TPWRS.2017.2734563.

[10] Google Cloud. *MLOps: Continuous delivery and automation pipelines in machine learning*. Google Cloud Architecture Center whitepaper. Defines a three-level MLOps maturity model (Level 0 manual, Level 1 ML pipeline automation, Level 2 CI/CD pipeline automation). 2024. URL: https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning.

[11] Per Runeson and Martin Höst. “Guidelines for conducting and reporting case study research in software engineering”. In: *Empirical Software Engineering* 14.2 (2009), pp. 131–164. DOI: 10.1007/s10664-008-9102-8.

[12] Eric Breck et al. “The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction”. In: *2017 IEEE International Conference on Big Data (Big Data)*. 2017, pp. 1123–1132. DOI: 10.1109/BigData.2017.8258038.

[13] Odd Erik Gundersen and Sigbjørn Kjensmo. “State of the Art: Reproducibility in Artificial Intelligence”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018. DOI: 10.1609/aaai.v32i1.11503.

[14] Ananth Mahadevan and Michael Mathioudakis. “Cost-aware retraining for machine learning”. In: *Knowledge-Based Systems* 293 (2024), p. 111610. DOI: 10.1016/j.knosys.2024.111610.

A Glossary

LEAR Lasso Estimated AutoRegressive ensemble for day-ahead electricity price forecasting, the primary baseline model from Lago et al. [3].

DNN Deep neural network. In this chapter, a multi-layer perceptron ensemble whose final prediction is the arithmetic mean of `n_dnn_ensemble` replicates trained with distinct random seeds.

EPF Electricity price forecasting: the task of predicting day-ahead wholesale electricity prices for a given bidding zone from historical prices and exogenous signals.

ENTSO-E European Network of Transmission System Operators for Electricity, the body that operates the Transparency Platform API from which the platform ingests price, load, and generation data.

Lasso Least Absolute Shrinkage and Selection Operator, an L1-regularised linear regression that simultaneously shrinks coefficients and performs automatic feature selection.

TPE Tree-structured Parzen Estimator, the Bayesian optimisation sampler used by Optuna when running `forecast-cli tune`.

MLOps Machine Learning Operations: the set of practices for reliably building, deploying, and operating machine learning systems in production.

ReplacingMergeTree A ClickHouse storage engine that deduplicates rows sharing the same sort key during background merge, used throughout the platform to make re-insertion safe under the platform's append-only write semantics.

Tailscale A mesh VPN built on WireGuard used here to grant the GitHub Actions runner ephemeral SSH access to the deploy host without exposing any port on that host to the public internet.

Loki Grafana Labs' log aggregation system, used here as the structured-log backend for all Prefect flows and infrastructure modules via direct Python push through `python-logging-loki`.

Evidently An open-source library for data and model drift detection, used here in in-process mode inside the Prefect drift-detection flow to apply its `DataDriftPreset`, which auto-selects a drift test (Kolmogorov–Smirnov or Wasserstein, by sample size) and compares a 90-day reference window against a 7-day current window.

B ML Test Score Worksheet

The full per-item ML Test Score worksheet of Breck et al. [12] follows. Each item scores 1 when the capability exists and is automated, tested, or repeatable, 0.5 when it exists but is manual, partial, or ad-hoc, and 0 when it is absent or genuinely not applicable to the domain, with one line of evidence pointing into the repository. The section sums are Data 2.5, Model development 3.0, ML infrastructure 4.0, and Monitoring 4.0. The final score is their minimum, **2.5** (§6.1).

Table B.1: Section 1: Data tests (sum 2.5 / 7).

#	Test	Score	Evidence
1	Feature expectations in a schema	1	`enforce_schema()` in `domain/curves.py`, raises on dtype/column mismatch, tested in `tests/domain/`
2	All features are beneficial	0	No systematic ablation documenting each feature's contribution (`forecast-cli ablate` exists but was not used for this)
3	No feature costs too much	0	No per-transformer latency/compute-cost analysis, no documented cost threshold
4	Features meet meta-level requirements	0	No regulatory/contractual/fairness requirements apply to wholesale price data
5	Pipeline has privacy controls	0	ENTSO-E data is public market data with no PII, n/a domain, scored 0
6	New features added quickly	0.5	`FeatureTransformer` ABC documented in `CONTRIBUTING.md`, manual process, no scaffold/SLA
7	All input feature code is tested	1	`tests/.../test_transformers.py` covers every transformer, runs in CI

Table B.2: Section 2: Model development tests (sum 3.0 / 7).

#	Test	Score	Evidence
1	Model specs reviewed and submitted	0.5	`configs/*.py` are first-class Git files under PR review, no automated spec-schema gate
2	Offline and online metrics correlate	0.5	Reconciliation computes the same MAE/RMSE/sMAPE on live predictions as backtesting, no formal correlation study
3	All hyperparameters tuned	0.5	DNN tuned with 6 HPs via Optuna, LEAR λ cross-validated, windows from the paper, Transformer partly manual
4	Impact of model staleness known	0.5	Daily refit bounds staleness < 24 h, no documented degradation experiment at 48 h / 7 d
5	A simpler model is not better	0.5	LEAR vs DNN vs Transformer tracked, rMAE vs weekly-naive, not an automated gate
6	Model quality sufficient on slices	0.5	Per-area backtests, `compute_metrics_per_hour`, no automated per-slice gate
7	Model tested for inclusion (fairness)	0	n/a domain, no demographic groups, scored 0

Table B.3: Section 3: ML infrastructure tests (sum 4.0 / 7).

#	Test	Score	Evidence
1	Training is reproducible	0.5	`experiment_runs` carries `git_sha` + data windows + `model_artifact_id`, LEAR deterministic, GPU non-determinism not suppressed
2	Model specs are unit tested	1	`test_models.py`, `test_transformer_model.py`, `test_dnn_ensemble.py` in CI
3	Full pipeline integration tested	1	LEAR smoke test plus full Compose-stack smoke in `pr_smoke.yml`
4	Quality validated before serving	0	No quality gate before predictions are written to `experiment_predictions`
5	Debuggable by observing steps	0.5	Loki structured logging, W&B per-epoch loss, no per-layer step-through
6	Tested via a canary process	0.5	`cd_deploy` `smoke_run` triggers a forecast post-deploy, checks completion, not quality
7	Quick and safe rollback	0.5	Artifact blobs in `model_artifacts`, image rollback via build + restart, no automated rollback command

Table B.4: Section 4: Monitoring tests (sum 4.0 / 7).

#	Test	Score	Evidence
1	Dependency changes notified	0	`uv.lock` tracked in Git, no Dependabot/Renovate notification
2	Data invariants hold train & serve	0.5	`enforce_schema` at construction, shared preprocessor, not independently monitored/alerted
3	Train/serve features compute alike	1	A single `DataPreprocessor` used by both `BacktestRunner` and `LiveRunner`, a structural guarantee
4	Models are not too stale	0.5	Daily refit < 24 h, Grafana `data-staleness` alert on inputs, no dedicated model-age alert
5	The model is numerically stable	0.5	`asinh` scaler, Transformer gradient clipping, no automated NaN/inf assertion on outputs
6	Compute has not regressed	0.5	`training_duration_seconds` tracked per run and in W&B, no automated threshold alert
7	Prediction quality not regressed	1	`_maybe_alert_accuracy_degradation` (MAE $> 1.5\times$ 30-day median) and the `reconciliation-missed` rule, both automated

A reproducible, monitored electricity-price-forecasting platform, together with a faithful, significance-tested comparison of the benchmark LEAR, DNN, and Transformer models on current ENTSO-E market data.

Technical
University of
Denmark

[Building / street]
2800 Kgs. Lyngby
Tlf. 4525 1700

www.dtu.dk